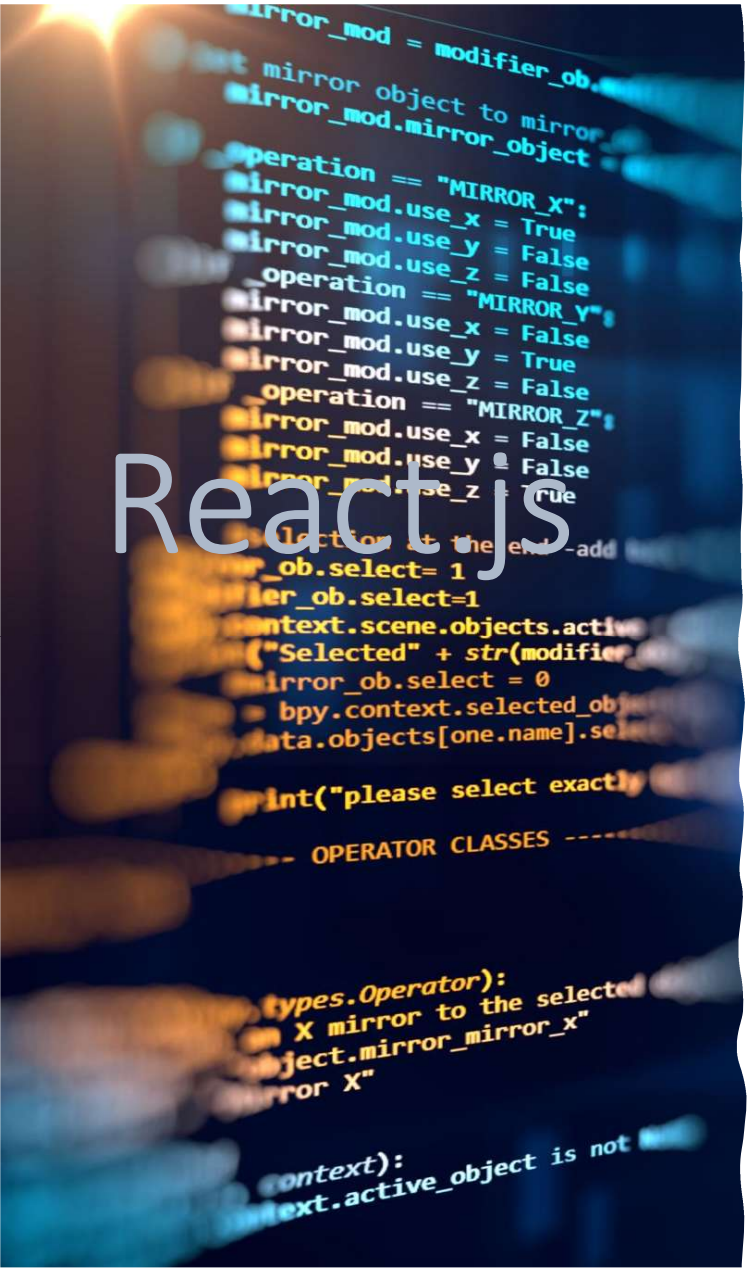
React.js

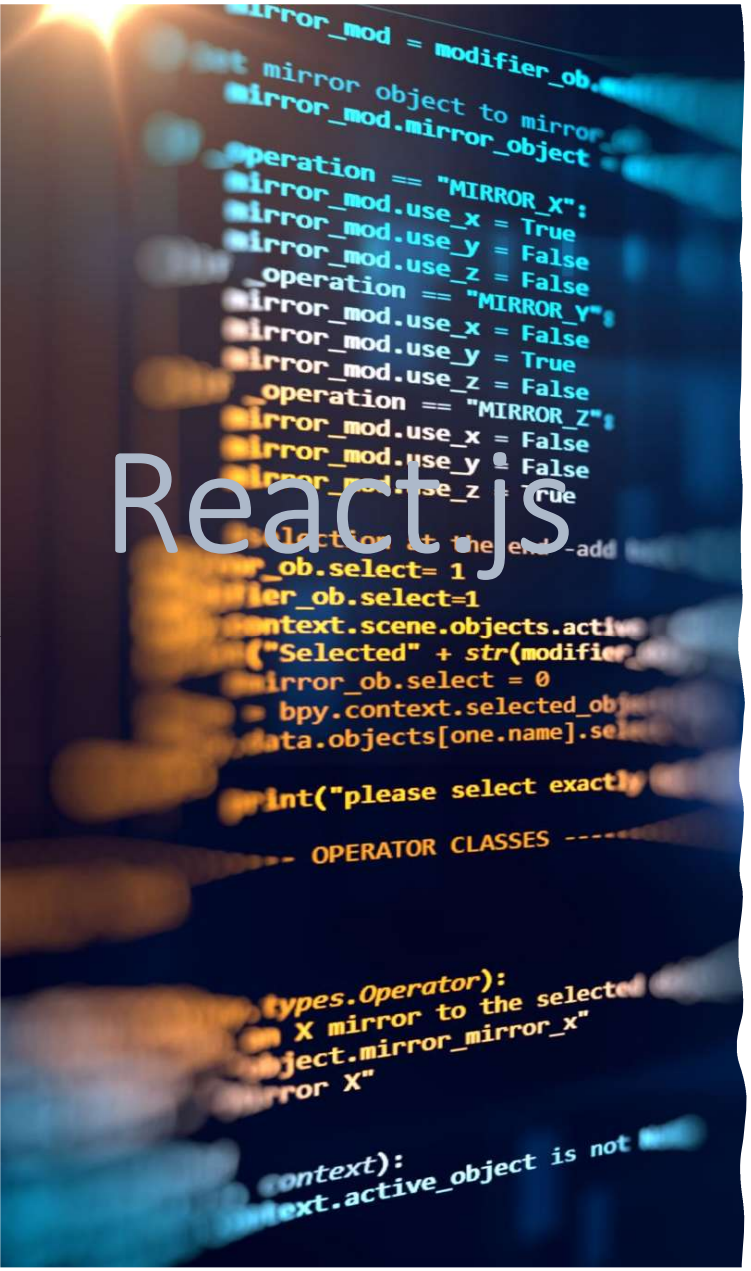
React.JS

NodeJs



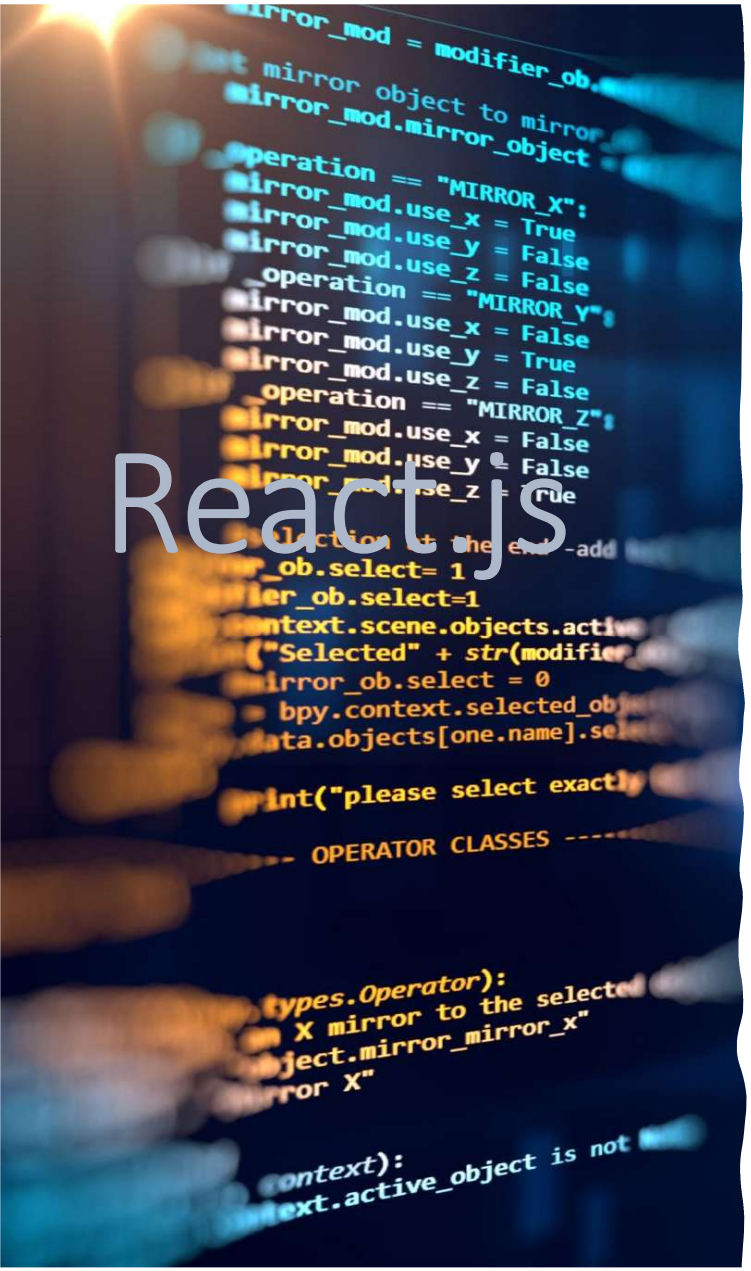
Qu'est-ce que React.js ?

- React.js, souvent simplement appelé React, est une bibliothèque JavaScript open-source pour la construction d'interfaces utilisateur.
- Développé par Facebook et maintenu par une communauté de développeurs, React se concentre sur la création d'interfaces utilisateur efficaces et interactives pour des applications web et mobiles.
- React permet de construire des applications en utilisant de petits blocs réutilisables appelés "composants". Chaque composant encapsule son propre état et logique, permettant une organisation modulaire et maintenable du code.



Importance et utilisation dans le développement web

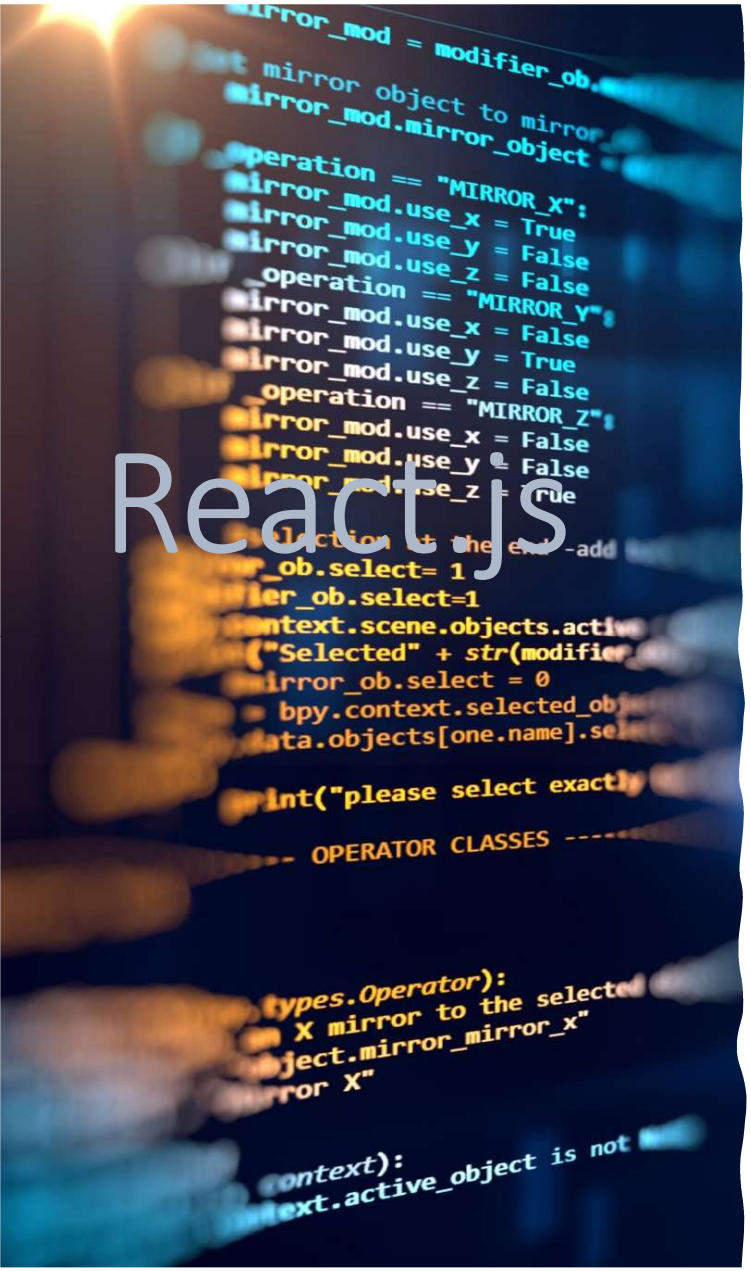
- React est connu pour sa flexibilité et sa performance, grâce à des concepts comme le Virtual DOM (Document Object Model), qui optimise les mises à jour de l'interface utilisateur.
- Il facilite la création d'applications web monopages (SPA - Single Page Applications) et d'applications mobiles (via React Native).
- La nature composante de React permet de facilement gérer l'état et les interactions au sein de l'application, rendant le développement plus intuitif et efficace.



Utilisation de React.js

- React.js est utilisé par de grandes entreprises comme Facebook, Instagram, Airbnb et Netflix.

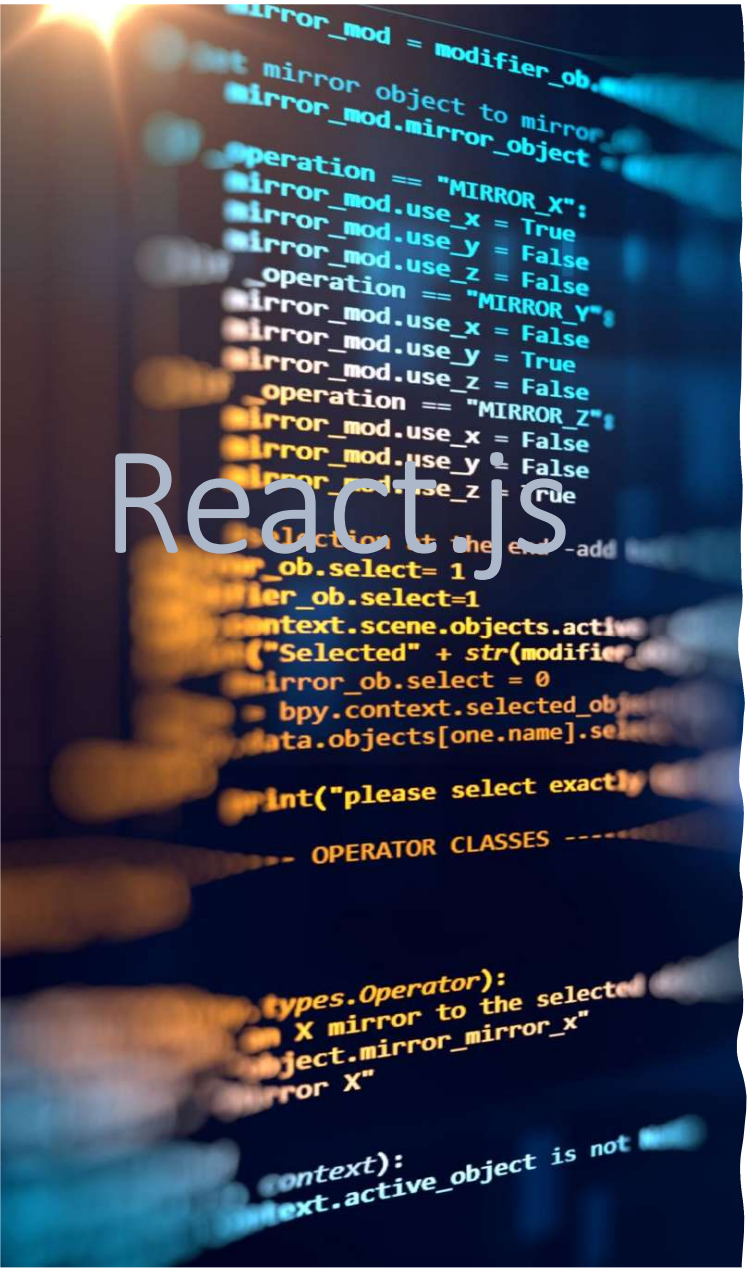
- Il est également populaire parmi les développeurs individuels grâce à sa flexibilité, sa performance et sa communauté active.



Les composants

- React.js est basé sur le concept de "composants".
- Un composant est une unité indépendante de code qui gère sa propre logique et son propre rendu.
- Les composants peuvent être réutilisés et combinés pour créer des interfaces utilisateur complexes.

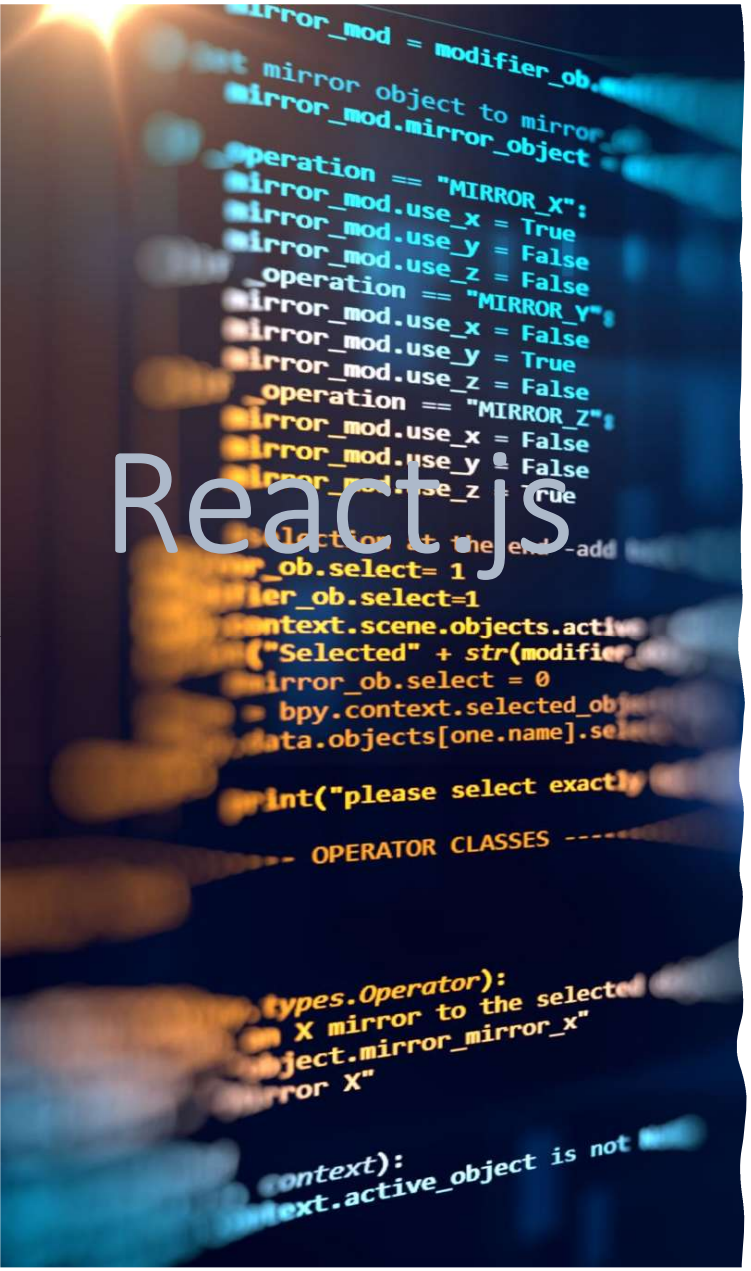
```
function Welcome(props) {
    return <h1>Bonjour, {props.name}</h1>;
  }
```



Le Virtual DOM

- React.js utilise une technique appelée Virtual DOM pour améliorer les performances.

- Au lieu de mettre à jour tout le DOM à chaque modification, React.js crée une copie du DOM, y apporte les modifications nécessaires, puis met à jour le DOM réel avec les différences minimales nécessaires.

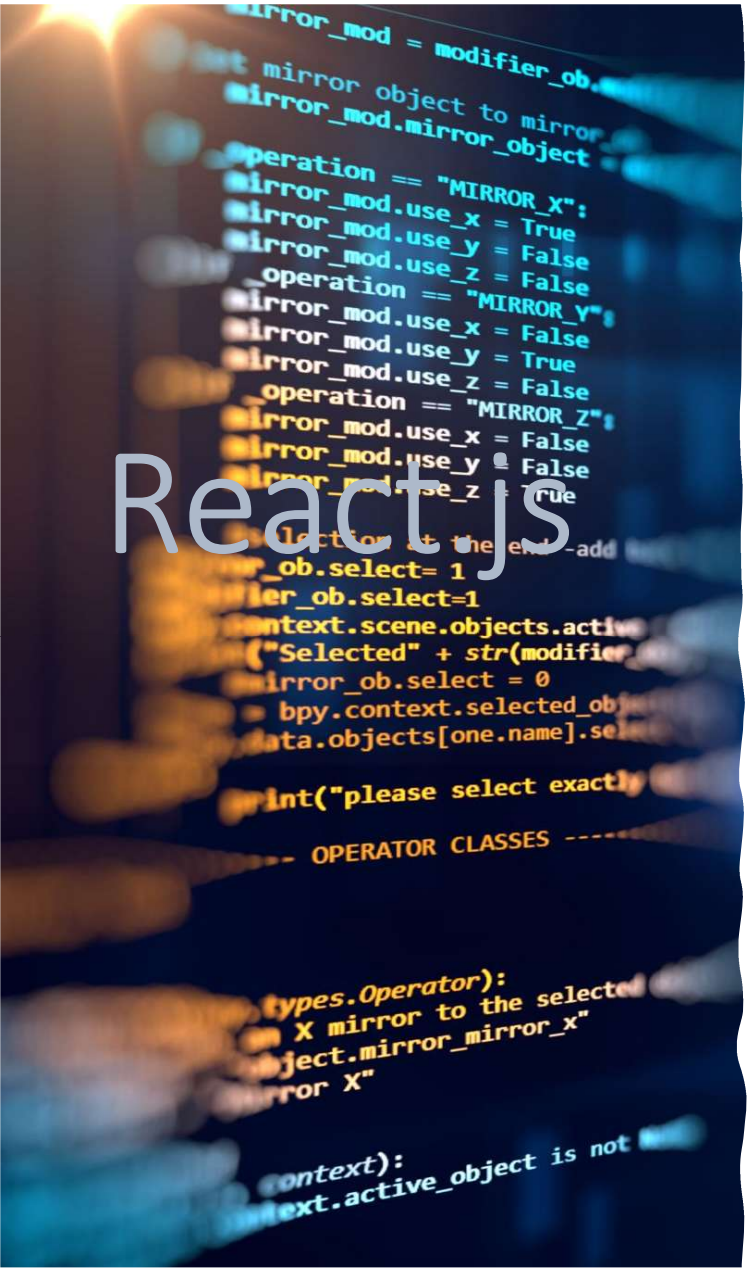


JSX

- React introduit une syntaxe appelée **JSX** qui permet d'écrire du code qui ressemble à du HTML dans le JavaScript. Cela rend le code plus lisible et plus facile à comprendre.

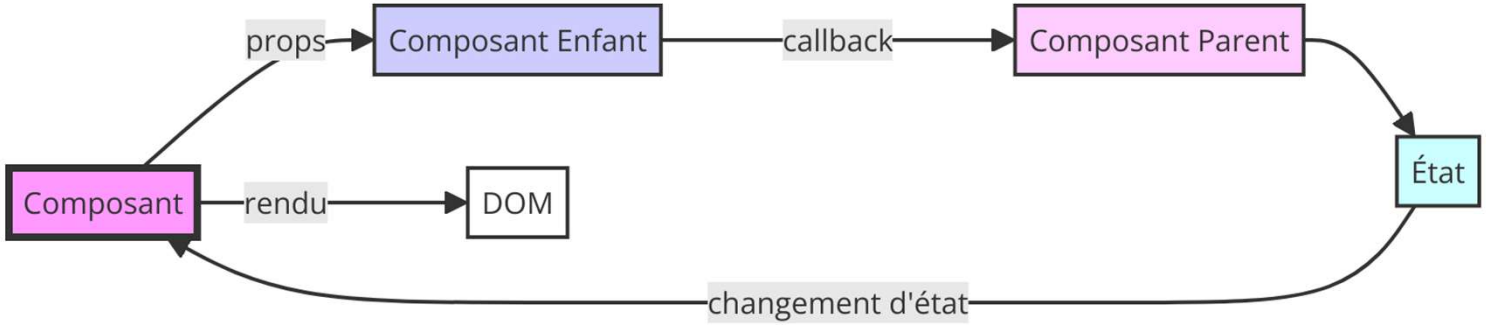
```
const name = 'Sarah';
const element = <h1>Bonjour, {name}</h1>;
```

```
function Welcome(props) {
  return <h1>Bonjour, {props.name}</h1>;
}
```

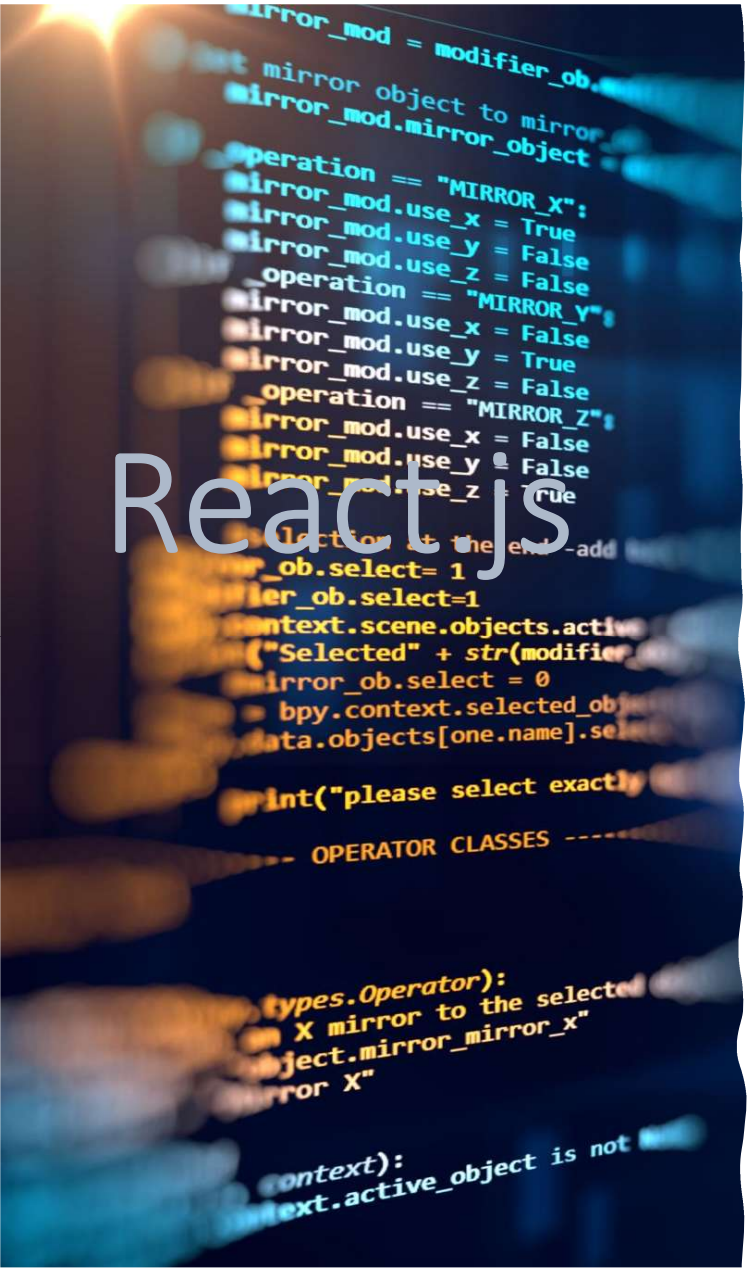


Flux de données unidirectionnel

- React suit un modèle de flux de données unidirectionnel. Cela signifie que les données dans une application React sont transmises des composants parents aux composants enfants via des "props". Cela rend le flux de données plus prévisible et plus facile à comprendre.



- L'état (state) est défini dans les composants parents et est transmis aux composants enfants via des props.
- Les interactions de l'utilisateur déclenchent des événements qui invoquent des fonctions callback, modifiant ainsi l'état du composant parent.
- Quand l'état change, React met à jour l'interface utilisateur en ne modifiant que les parties nécessaires du Document Object Model (DOM), conduisant à un rendu efficace.

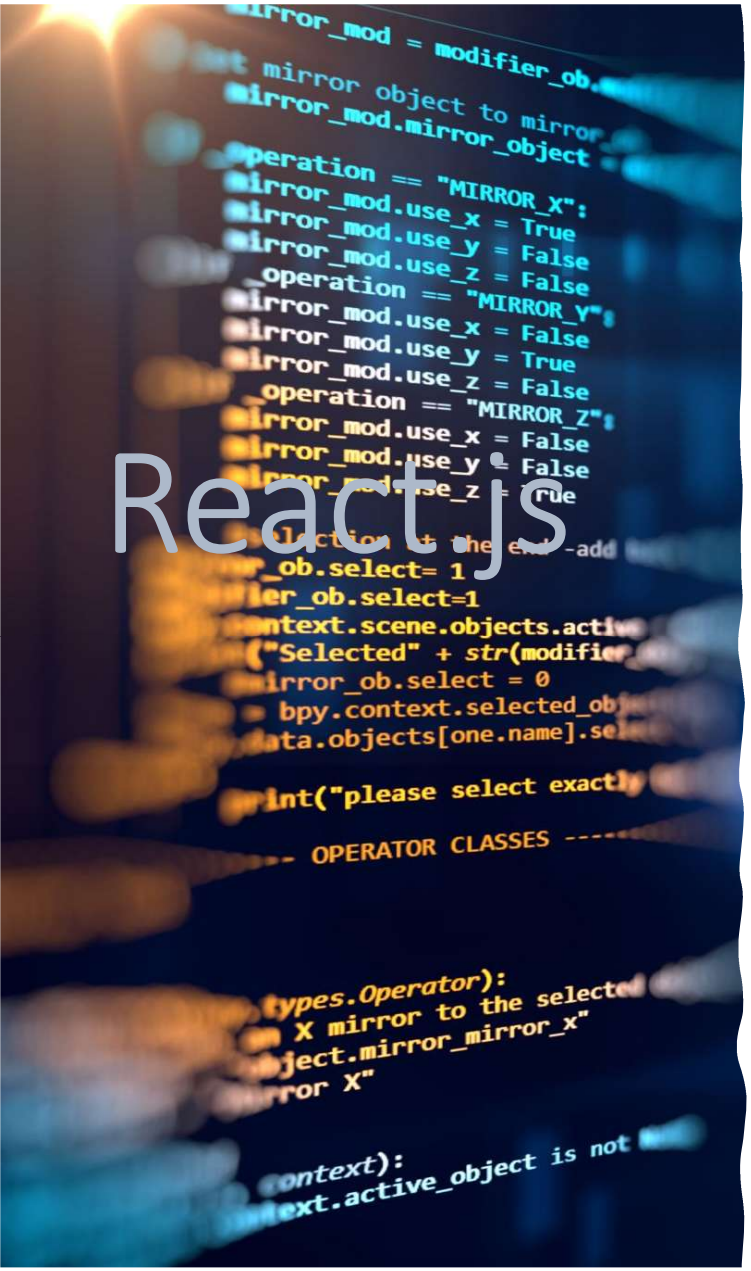


Création d'un nouveau projet React.js

- **create-react-app**, un outil qui configure automatiquement un nouvel environnement de projet React.js.
- Création d'un nouveau projet en utilisant la commande:

  npx create-react-app nom_du_projet

  cd nom_du_projet
  npm start

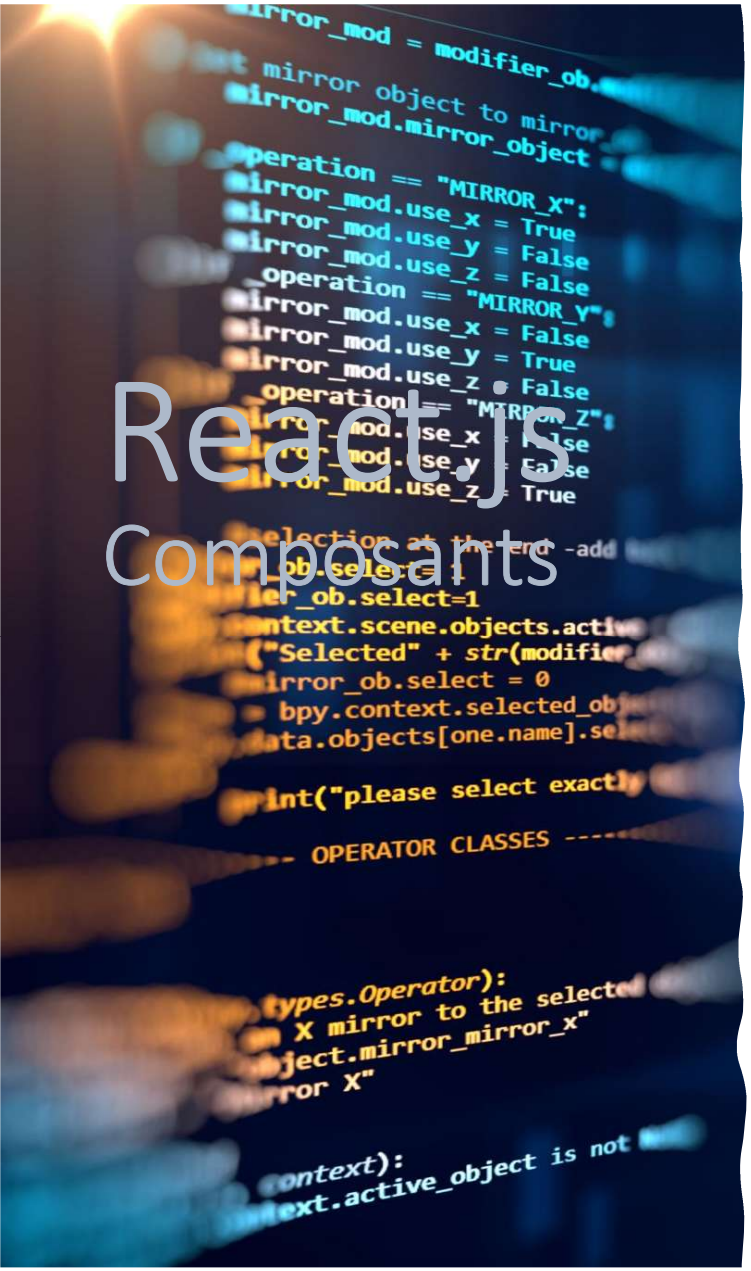


Structure du projet React.js

- **Dossier node_modules** : Ce dossier contient toutes les bibliothèques et dépendances que le projet utilise. Ces dépendances sont définies dans le fichier package.json et sont installées lors de l'exécution de npm install.
- **Dossier public** : Ce dossier contient le fichier index.html, qui est la page sur laquelle l'application React sera rendue. Il peut également contenir d'autres ressources statiques comme des images ou des fichiers CSS.
- **Dossier src** : Ce dossier contient tout le code source de l'application React. C'est ici que se créent les composants React. Par défaut, il contient quelques fichiers comme App.js (le composant principal de l'application), index.js (le point d'entrée de l'application), et App.css (pour les styles de l'application).
- **Fichier package.json** : Ce fichier contient des informations sur le projet et les dépendances qu'il utilise. On peut également y ajouter des scripts pour automatiser certaines tâches, comme le démarrage de l'application ou l'exécution de tests.
- **Fichier .gitignore** : Ce fichier indique à Git quels fichiers ou dossiers il doit ignorer. Par défaut, il ignore le dossier node_modules, car il n'est pas nécessaire de suivre les dépendances du projet dans Git.
- **Fichier README.md** : Ce fichier contient des informations sur le projet et comment l'utiliser. On peut y ajouter des instructions pour d'autres développeurs, des notes sur la conception de l'application, etc.

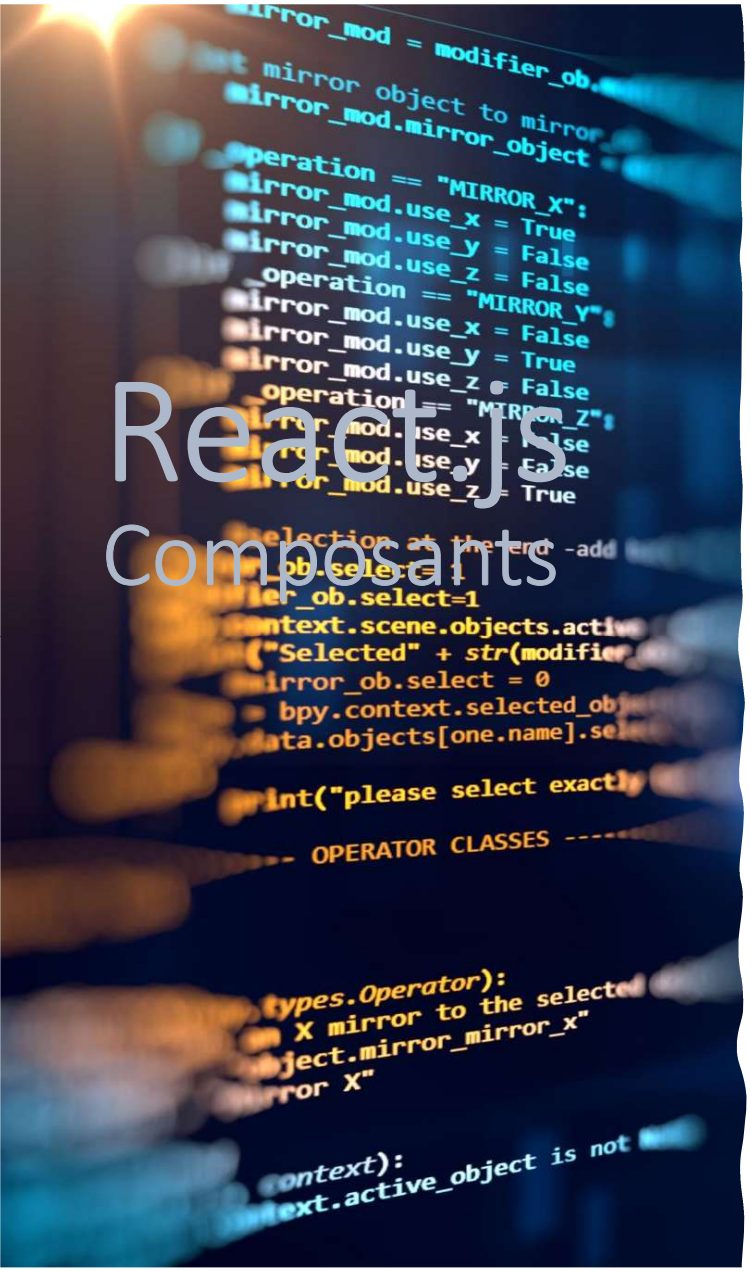
Ces fichiers et dossiers constituent la base du projet React.js. À mesure que le projet grandit, on peut ajouter d'autres fichiers et dossiers selon les besoins.

React.js
Composants React



Les composants React

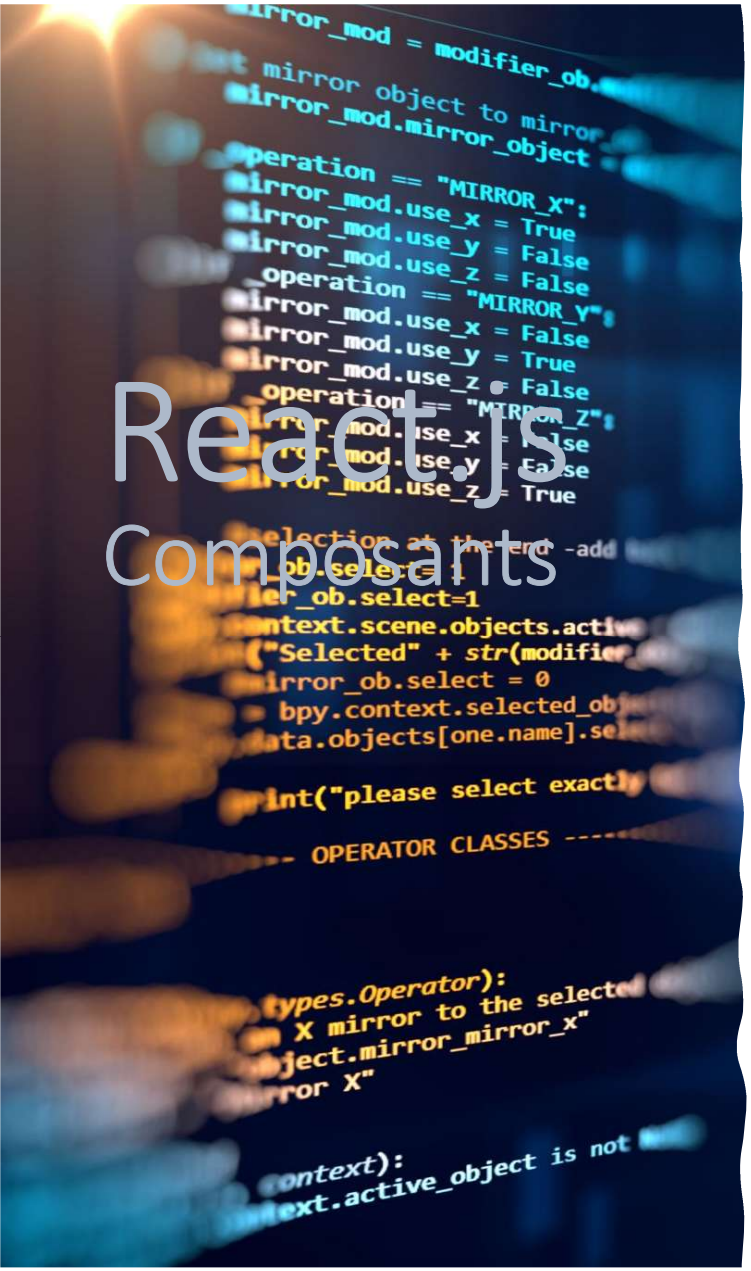
- Un composant React est une unité indépendante de code qui retourne un élément React (généralement via JSX) à afficher dans l'interface utilisateur.
- Les composants sont le cœur de toute application React. Ils permettent de diviser l'interface utilisateur en parties indépendantes et réutilisables.
- Composants de Classe: Ces composants sont des classes ES6 qui étendent React.Component et incluent des méthodes de cycle de vie pour exécuter du code à des moments clés.
- Composants Fonctionnels: Avec l'introduction des Hooks dans React 16.8, les composants fonctionnels peuvent utiliser des états et d'autres fonctionnalités React sans avoir à écrire une classe.



Exemple de Composant de Classe

```
class Welcome extends React.Component {
    render() {
      return <h1>Bonjour, {this.props.name}</h1>;
    }
  }
```

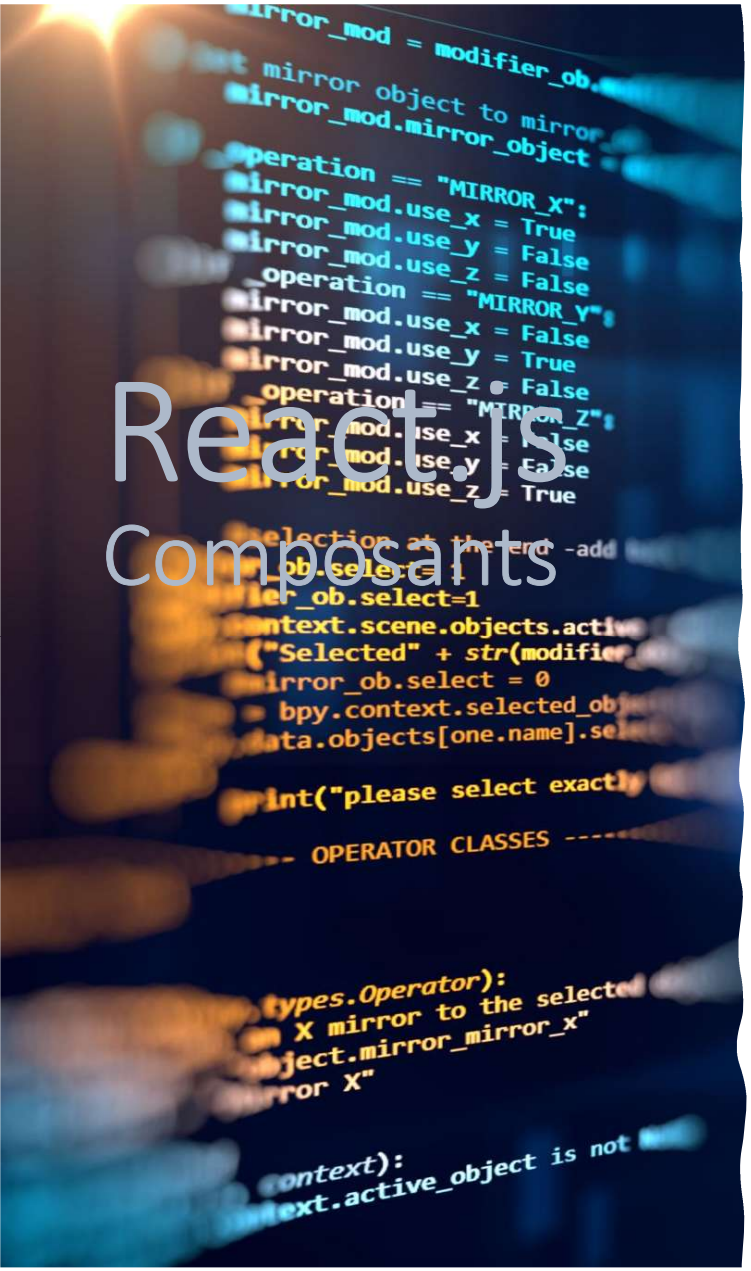
- Caractéristiques: Utilisation des méthodes de cycle de vie, état interne complexe, accès à this.



Exemple de Composant Fonctionnel

```
function Welcome(props) {
    return <h1>Bonjour, {props.name}</h1>;
  }
```

- Caractéristiques: Utilisation des méthodes de cycle de vie, état interne complexe, accès à this.

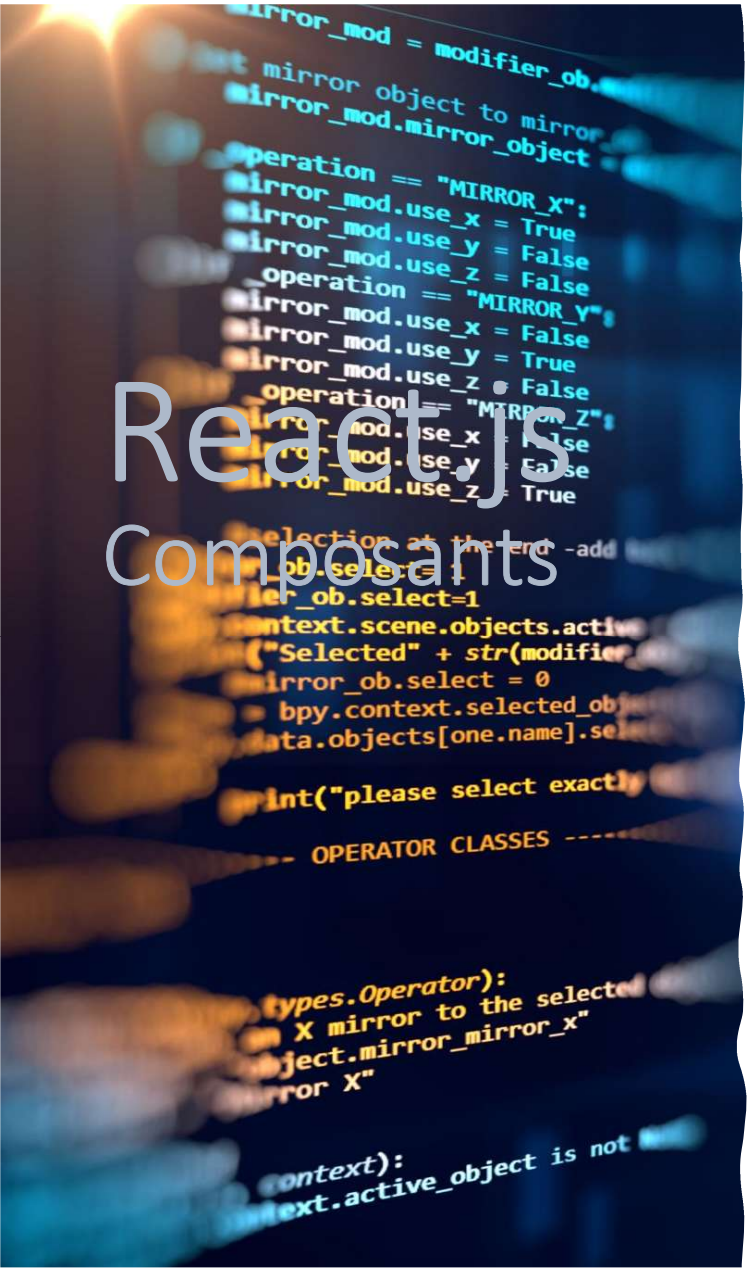


Composants fonctionnels

- Les composants fonctionnels sont des fonctions JavaScript qui prennent des props (propriétés) en entrée et retournent un élément React.
- Ils sont appelés "fonctionnels" parce qu'ils sont littéralement des fonctions JavaScript.
- Exemple de code d'un composant fonctionnel

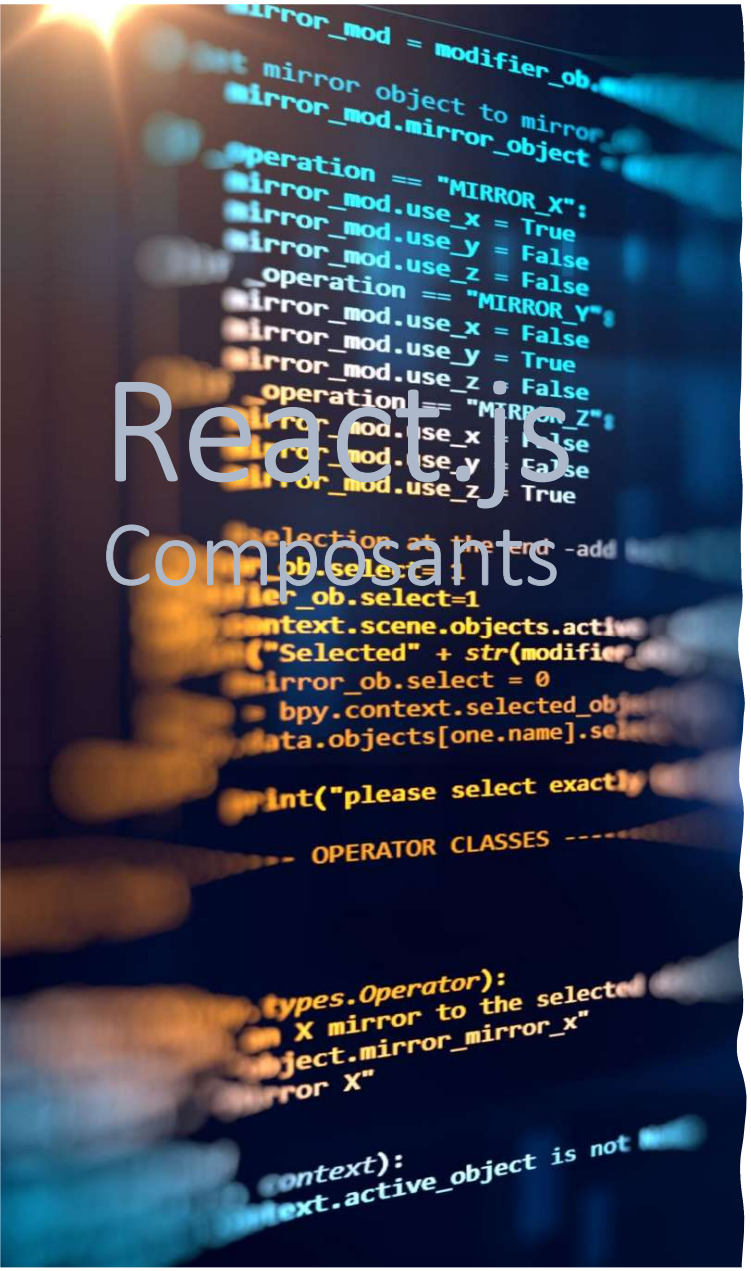
```
function Welcome(props) {
    return <h1>Bonjour, {props.name}</h1>;
  }
```

- Dans cet exemple, Welcome est un composant fonctionnel qui prend un objet props en entrée. Cet objet props contient une propriété name, qui est utilisée dans le rendu du composant.



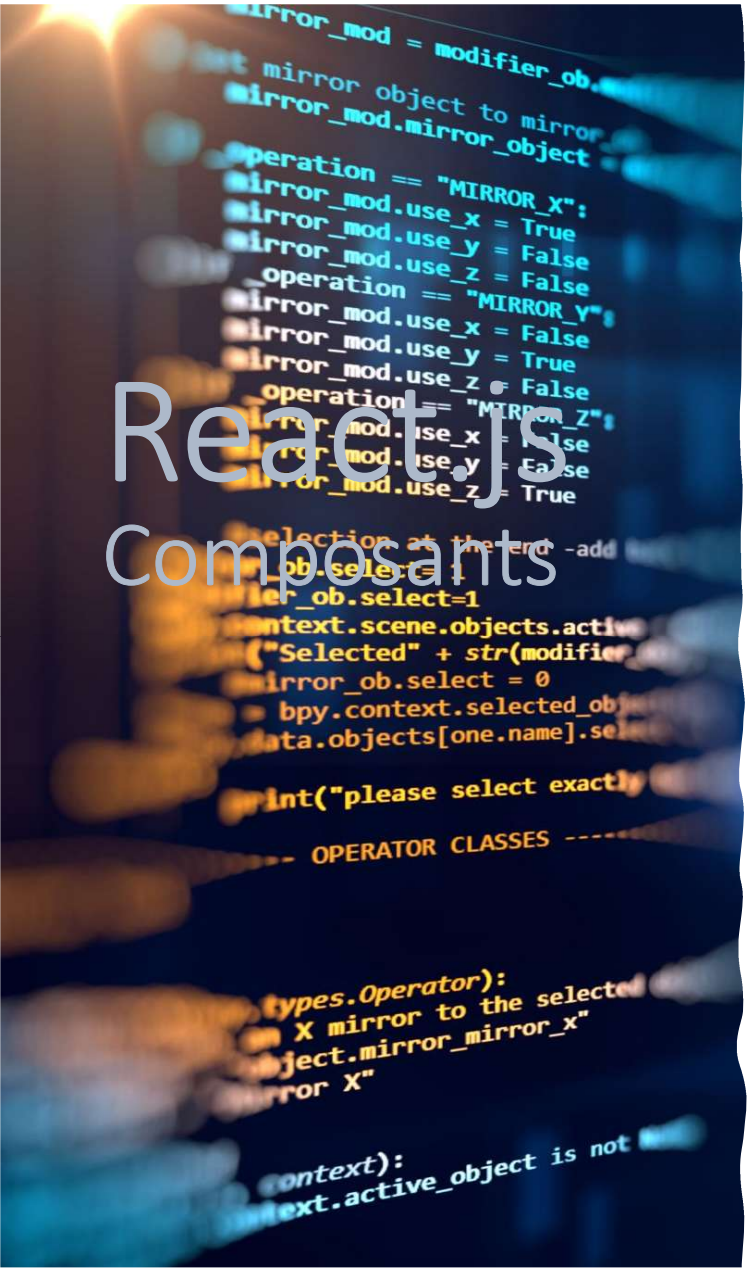
Exemple d'utilisation

```
import React from 'react';
// Composant de Classe
class WelcomeClass extends React.Component {
  render() {
    return <h1>Bonjour, {this.props.name} (Classe)</h1>;
  }
}
// Composant Fonctionnel
function WelcomeFunction(props) {
  return <h1>Bonjour, {props.name} (Fonctionnel)</h1>;
}
function App() {
  return (
    <div>
      <WelcomeClass name="Alice" />
      <WelcomeFunction name="Bob" />
    </div>
  );
}
export default App;
```



Composants fonctionnels vs composants de classe

- Similarités: Les deux peuvent recevoir des props et gérer des états (les fonctionnels avec useState).
- Différences:
- Syntaxe et Verbose: Les classes sont plus verbales; les fonctions sont plus succinctes.
- Utilisation des Hooks: Seulement dans les composants fonctionnels pour gérer l'état et les effets sans écrire une classe.
- Méthodes de Cycle de Vie: Disponibles uniquement dans les composants de classe.
- Tendance Actuelle: La tendance est à l'utilisation de composants fonctionnels en raison de leur simplicité et la puissance des Hooks.



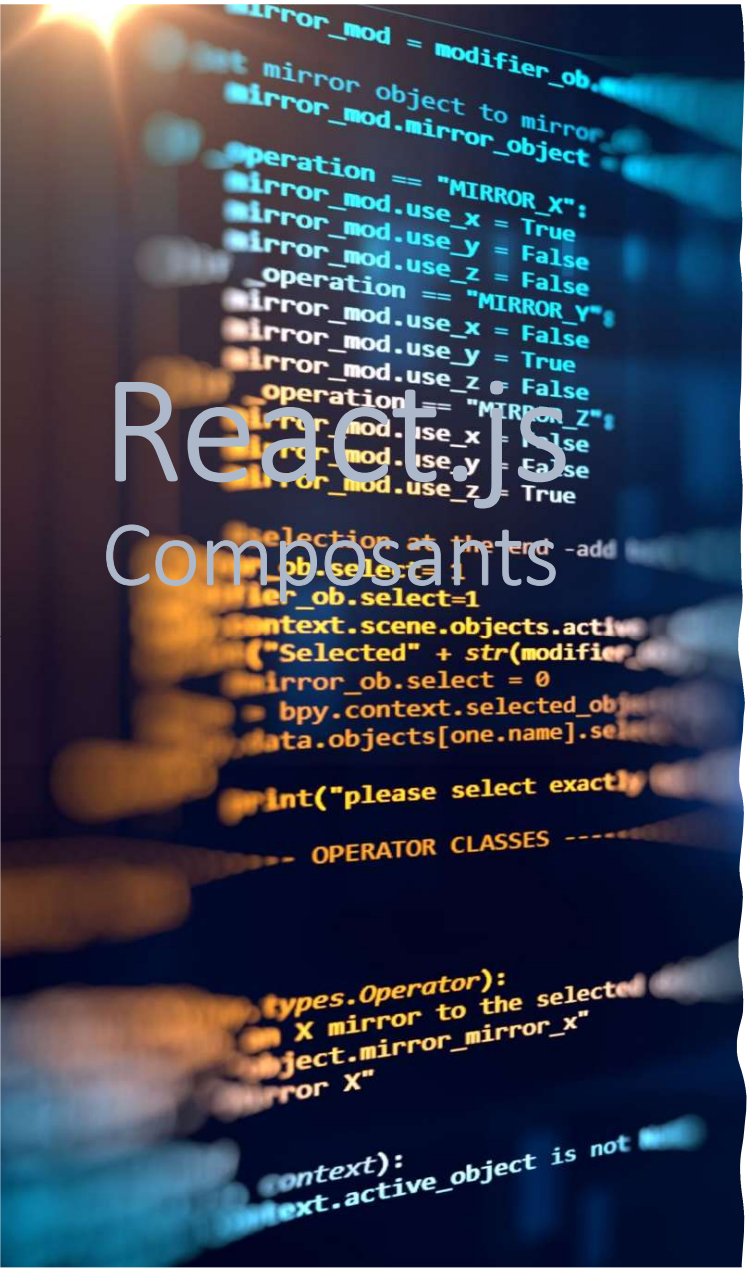
Les props ?

- Qu'est-ce que les props ?
  - Les props (propriétés) sont utilisées pour transmettre des données d'un composant parent à un composant enfant en React.
  - Les props permettent de configurer le comportement d'un composant en lui fournissant des données spécifiques.

- Utilisation des Props
  - Les props sont passées aux composants comme des attributs lors de leur utilisation.
  - Exemple :

    <ChildComponent prop1={valeur1} prop2={valeur2} />

- Accéder aux Props dans le Composant Enfant
  - Dans le composant enfant, les props peuvent être accédées via l'objet props.
  - Exemple : **props.nomProp** pour accéder à la valeur de la prop nommée "**nomProp**".



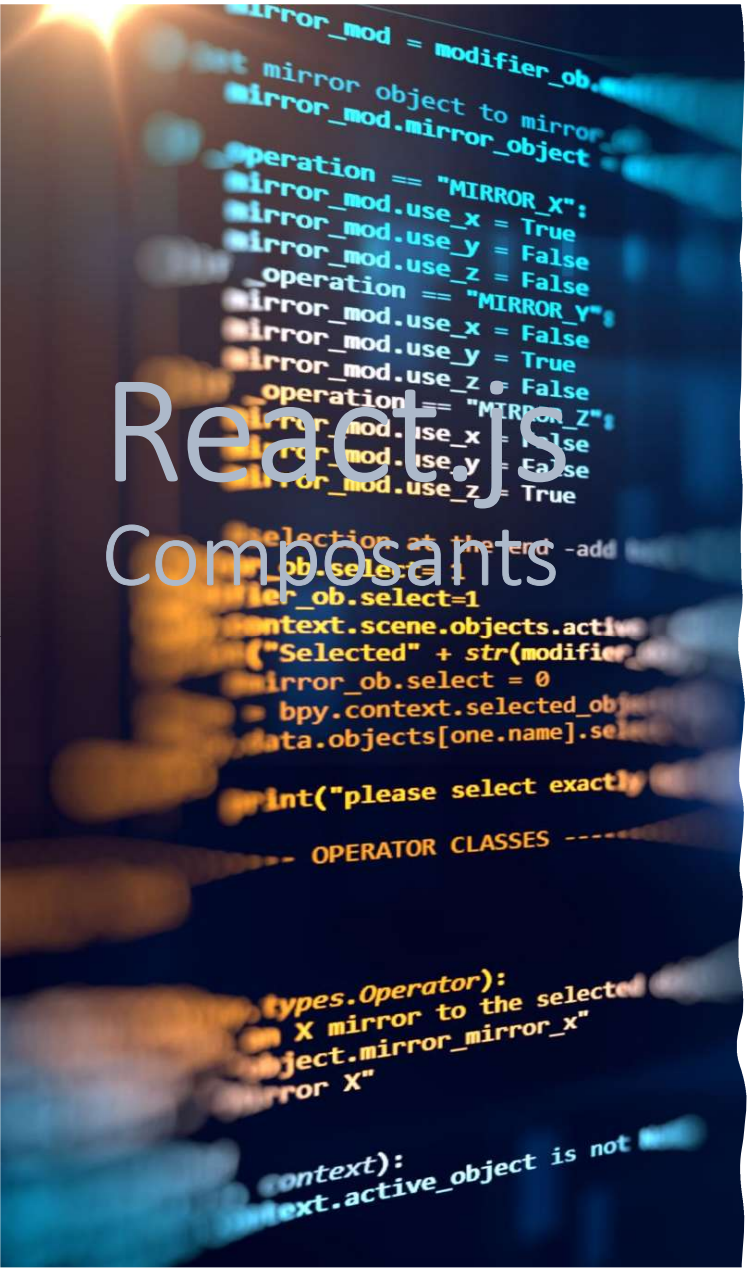
Les props ?

- Composant parent

```
function ParentComponent() {
  const name = "John Doe";
  const age = 30;

  return (
    <ChildComponent name={name} age={age} />
  );
}
```

- Composant enfant

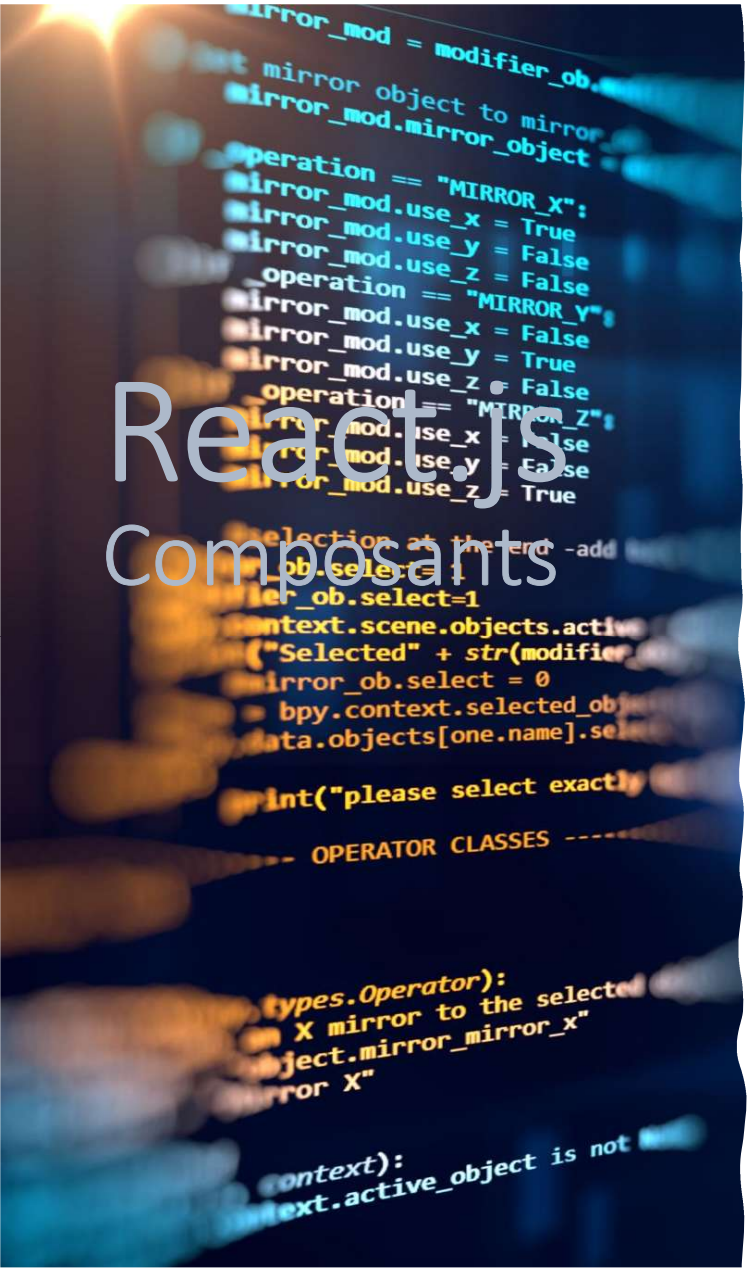
```
function ChildComponent(props) {
  return (
    <div>
      <h1>Je m'appelle {props.name}</h1>
      <p>J'ai {props.age} ans.</p>
    </div>
  );
}
```



Les props ?

- **Avantages des Props**
  - ➢Les props rendent les composants réutilisables en leur permettant d'accepter différentes données en fonction du contexte d'utilisation.
  - ➢Les props facilitent la communication entre les composants parent et enfant.

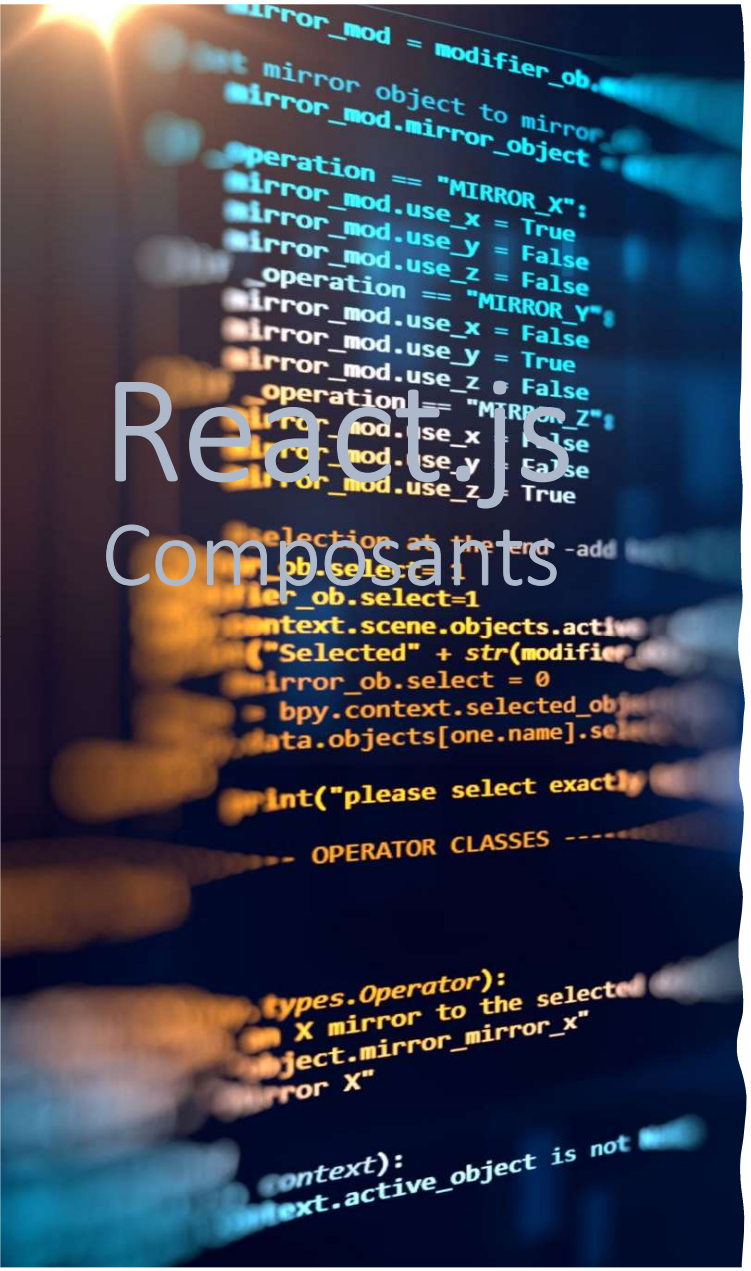
- **Bonnes Pratiques**
  - ➢Éviter de modifier directement les props dans le composant enfant, car elles sont considérées comme immuables.
  - ➢Si on doit modifier les données des props, on crée une copie locale et on effectue les modifications sur cette copie.



Introduction aux Hooks

- Les Hooks sont une fonctionnalité introduite dans React 16.8 qui permet d'utiliser l'état et d'autres fonctionnalités de React sans écrire une classe.

- Les Hooks permettent de réutiliser la logique d'état sans changer la hiérarchie des composants, ce qui rend le code plus facile à maintenir et à tester.

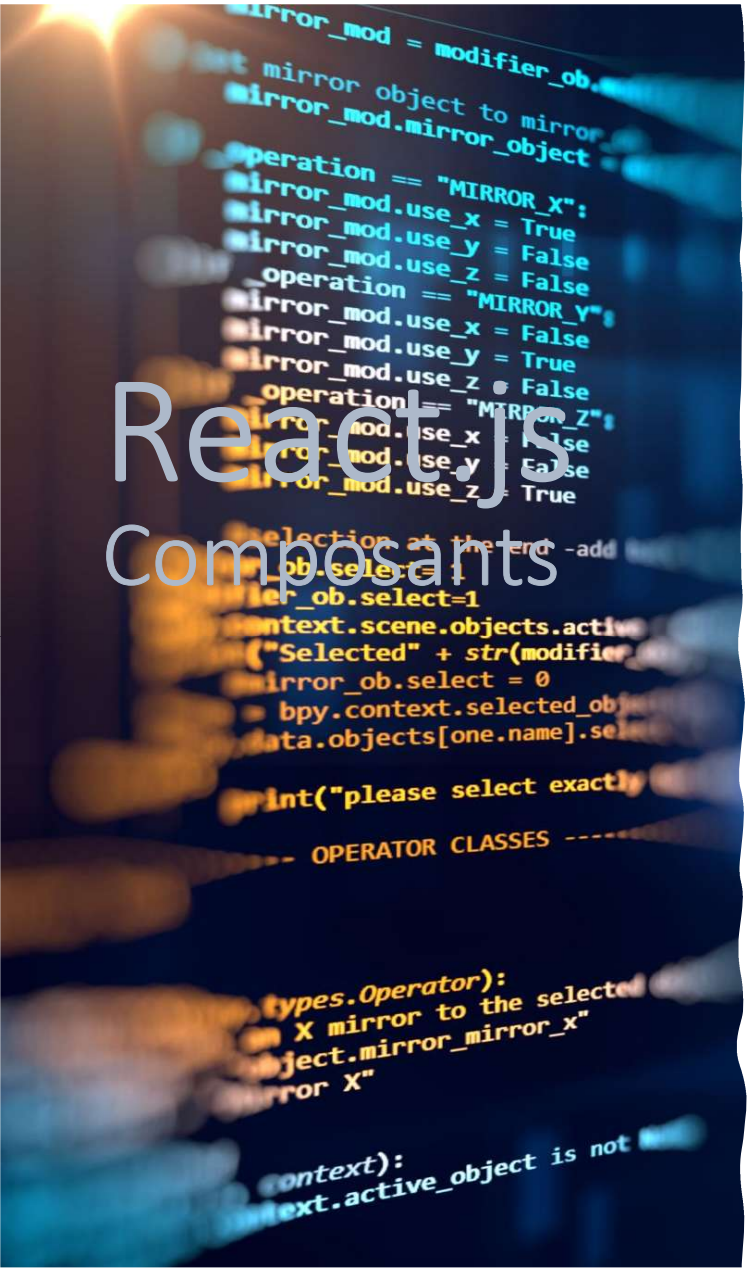


Utilisation des Hooks dans les composants fonctionnels

- Présentation du Hook useState, qui permet d'ajouter un état local à un composant fonctionnel.
- Exemple de code d'un composant fonctionnel utilisant useState

```
import React, { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Vous avez cliqué {count} fois</p>
      <button onClick={() => setCount(count + 1)}>
        Cliquez moi
      </button>
    </div>
  );
}
```

- Dans cet exemple, useState est utilisé pour créer une variable d'état count et une fonction pour la mettre à jour setCount. Le bouton utilise setCount pour augmenter la valeur de count chaque fois qu'il est cliqué.

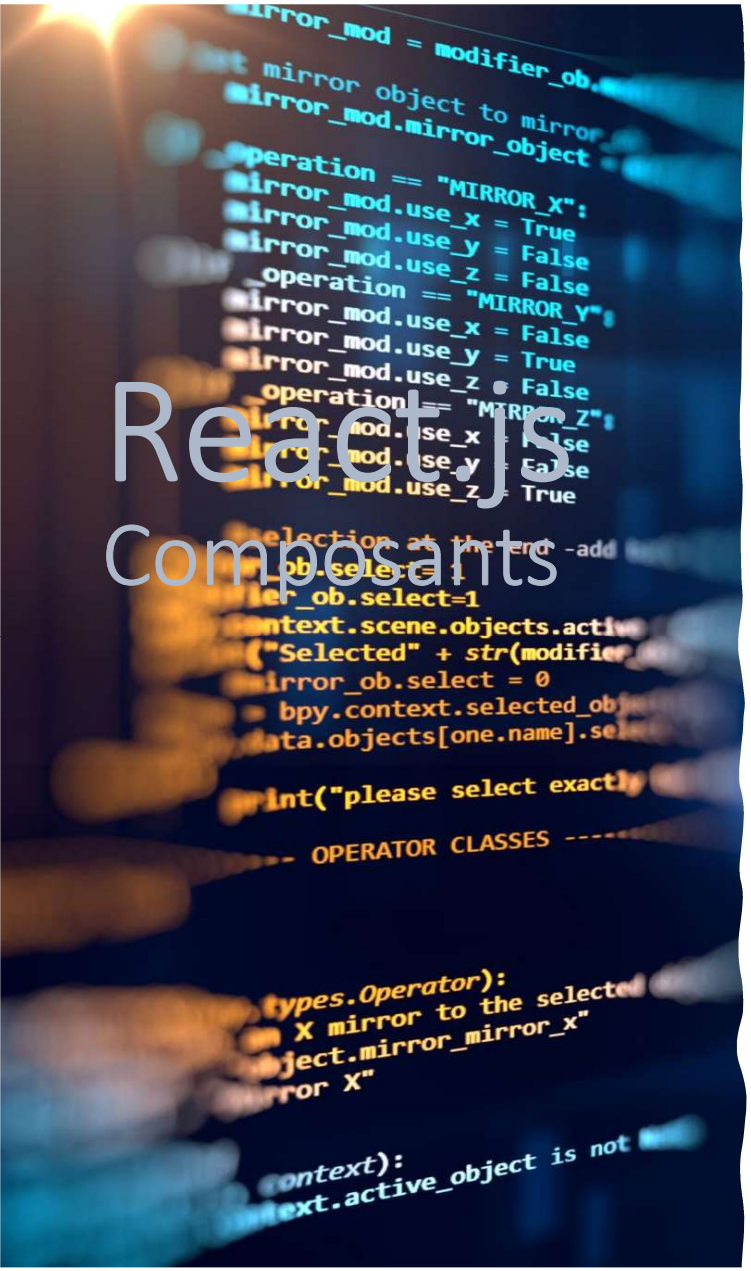


useEffect Hook

useEffect permet d'effectuer des effets de bord dans les composants fonctionnels. Les effets de bord sont des opérations qui affectent d'autres parties du système, comme la récupération de données, les abonnements ou le changement manuel du DOM React. **useEffect** est similaire aux méthodes **componentDidMount**, **componentDidUpdate**, et **componentWillUnmount** dans les classes.

```
import React, { useState, useEffect } from 'react';
function Example() {
  const [count, setCount] = useState(0);
  // Similaire à componentDidMount et componentDidUpdate :
  useEffect(() => {
    // Met à jour le titre du document en utilisant l'API du navigateur
    document.title = `Vous avez cliqué ${count} fois`;
  });
  return (
    <div>
      <p>Vous avez cliqué {count} fois</p>
      <button onClick={() => setCount(count + 1)}>
        Cliquez moi
      </button>
    </div>
  );
}
```


Dans cet exemple, l'effet met à jour le titre du document après que React a mis à jour le DOM.

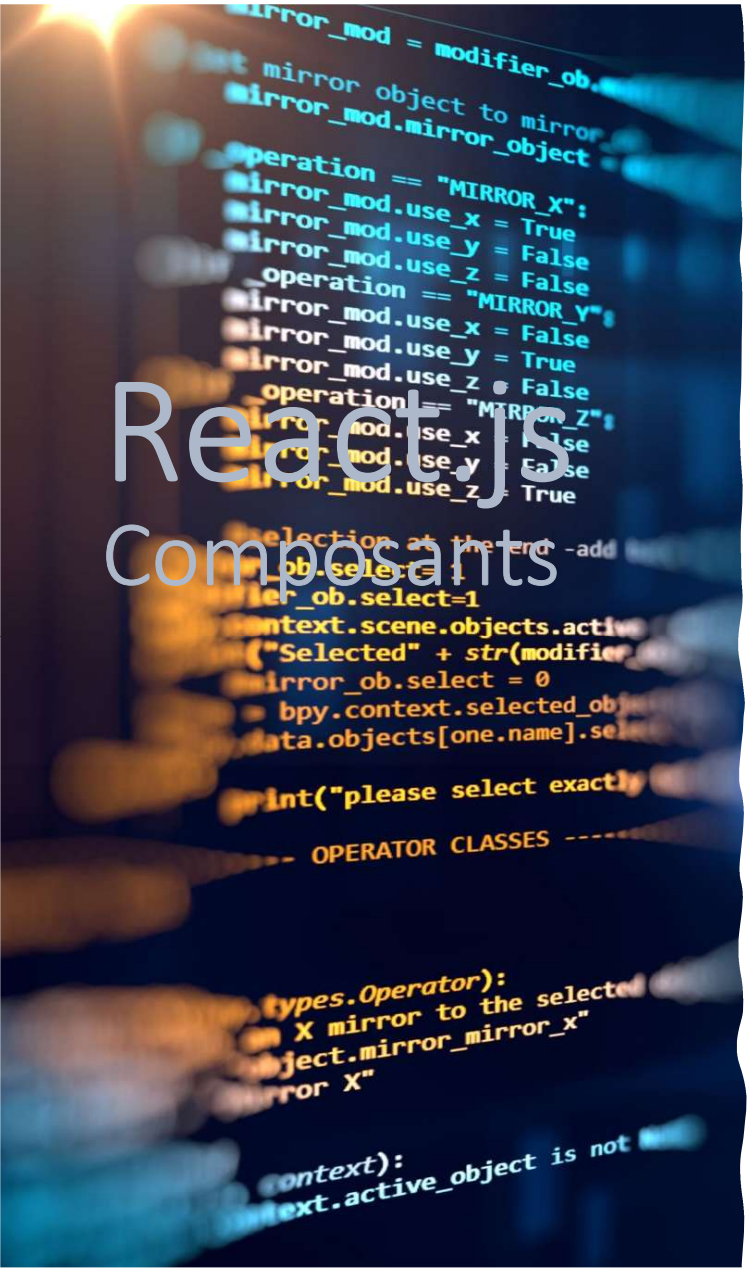


useEffect Hook

Le hook **useEffect** prend deux arguments : une fonction de rappel et un tableau de dépendances optionnel. La fonction de rappel est exécutée après chaque rendu du composant, à moins que les dépendances spécifiées dans le tableau de dépendances ne changent. Si les dépendances ne sont pas spécifiées, la fonction de rappel est exécutée après chaque rendu.

```
//Exécution d'une action après le rendu initial.
// la fonction de rappel est exécutée une seule fois après le rendu initial du
composant, car le tableau de dépendances est vide ([]).

const [count, setCount] = useState(0);

useEffect(() => {
    console.log('Le composant a été rendu');
  }, []);
```

```
// Exécution d'une action en réponse à un changement de dépendance
//  la fonction de rappel est exécutée uniquement lorsque la valeur de count change. Si
count reste inchangé, la fonction de rappel ne sera pas exécutée après les rendus
suivants.

const [count, setCount] = useState(0);

useEffect(() => {
  console.log('La valeur de count a changé');
}, [count]);
```

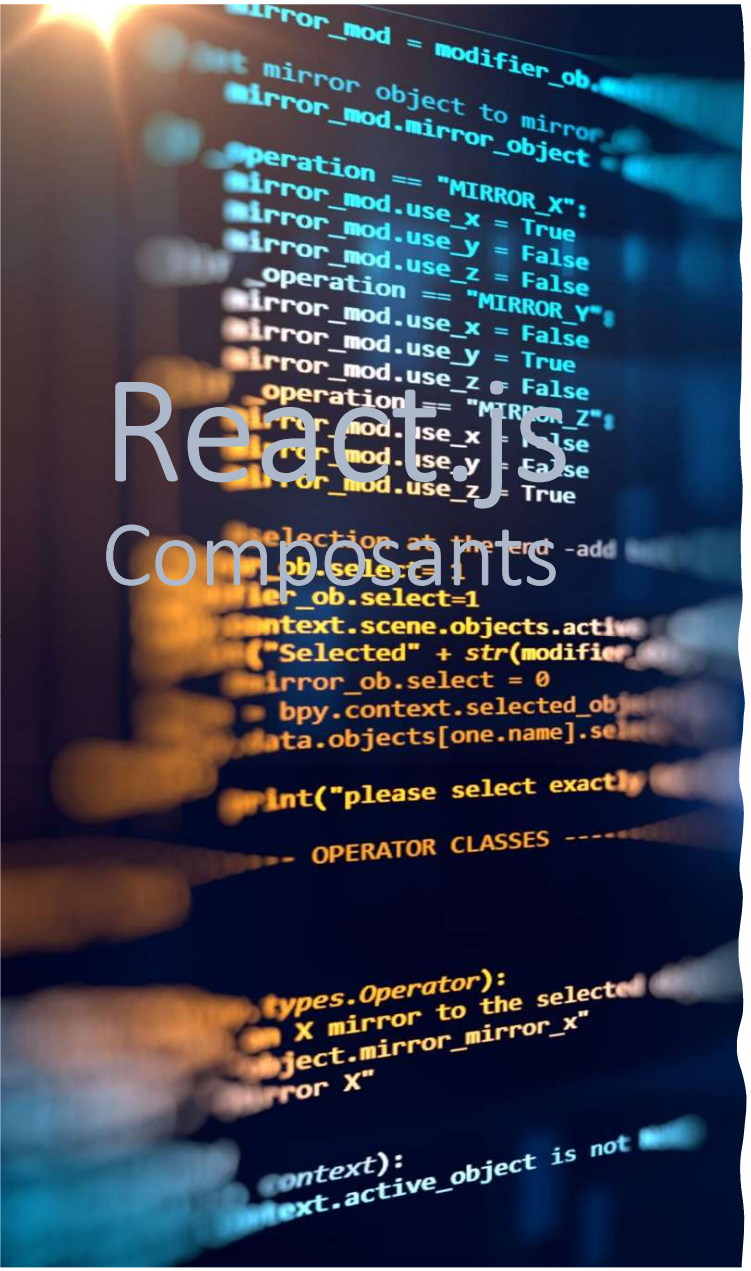


useContext Hook

useContext est utilisé dans React pour accéder à une valeur fournie par un contexte (Context) dans un composant fonctionnel. Il permet de récupérer la valeur du contexte et de s'abonner à ses mises à jour. Cela évite de devoir passer la valeur du contexte de manière explicite à travers les props entre les composants parents et enfants.

```
import React, { useContext } from 'react';
const ThemeContext = React.createContext('light');
function ThemedButton() {
  const theme = useContext(ThemeContext);
  return <button theme={theme}>Je suis un bouton stylisé</button>;
}
```

Dans cet exemple, **useContext** est utilisé pour passer la valeur du thème à ThemedButton.



useReducer Hook

useReducer permet de gérer des états complexes qui impliquent plusieurs sous-valeurs ou lorsque la logique de mise à jour de l'état est plus complexe. **useReducer** est également utile lorsque la performance est un problème, car il permet d'optimiser les mises à jour de composants qui se produisent en réponse à des événements de dispatch.

```react_counter.js
import React, { useReducer } from 'react';

// Fonction réducteur
const reducer = (state, action) => {
  switch (action.type) {
    case 'INCREMENT':
      return state + 1;
    case 'DECREMENT':
      return state - 1;
    default:
      return state;
  }
};

const Counter = () => {
  const [count, dispatch] = useReducer(reducer, 0);

  const increment = () => {
    dispatch({ type: 'INCREMENT' });
  };

  const decrement = () => {
    dispatch({ type: 'DECREMENT' });
  };

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={increment}>Increment</button>
      <button onClick={decrement}>Decrement</button>
    </div>
  );
};
```

Dans cet exemple, **useReducer** est utilisé pour gérer l'état d'un compteur, qui peut être incrémenté ou décrémenté.

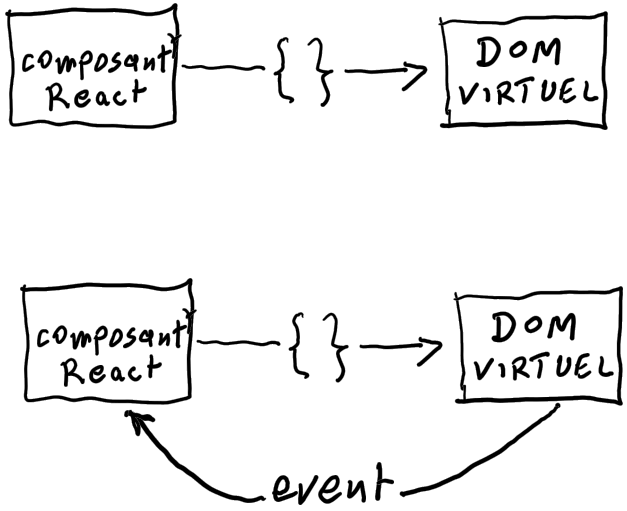
React.js

Gestion des événements utilisateur dans les composants React.

Gestion des événements

Dans React, les événements sont gérés en utilisant la syntaxe JSX et les gestionnaires d'événements.

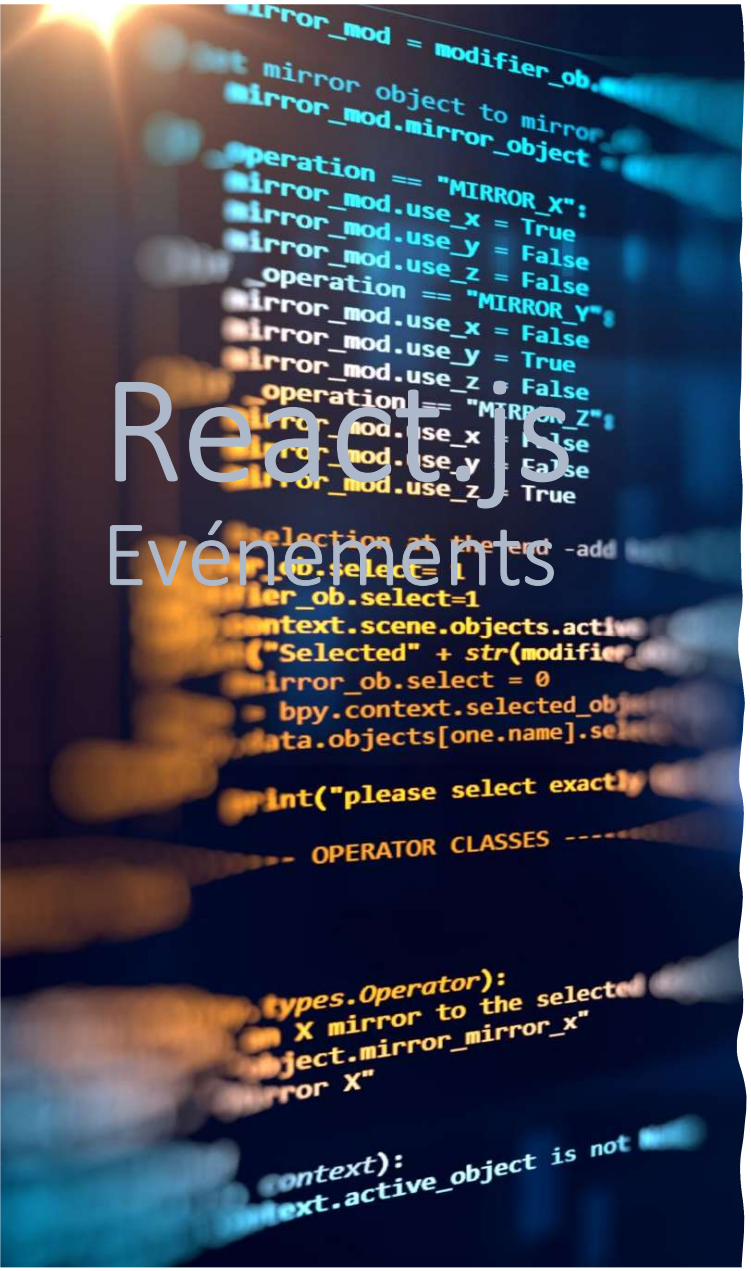
Les gestionnaires d'événements sont des fonctions qui sont exécutées en réponse à une action de l'utilisateur, telle que le clic sur un bouton ou la soumission d'un formulaire.



Gestion des événements

Exemple d'un composant avec un bouton et un gestionnaire d'événements.

```
import React, { useState } from "react";

function Button() {
  const handleClick = () => {
    console.log("Button clicked!");
  };

  return <button onClick={handleClick}>Click me</button>;
}
```



Gestion des événements

- Dans cet exemple, on a un composant Button avec un gestionnaire d'événements handleClick.
- Lorsque l'utilisateur clique sur le bouton, la fonction handleClick est appelée.
- On peut attacher un gestionnaire d'événements à des éléments JSX en utilisant l'attribut onClick et en lui passant la référence de la fonction de gestionnaire d'événements.

Gestion des événements

- Les gestionnaires d'événements peuvent effectuer diverses actions, telles que la mise à jour de l'état, l'appel à des API, la navigation vers d'autres pages, etc.
- Il est également possible d'accéder à des informations sur l'événement lui-même, comme les coordonnées du clic de la souris ou les données d'un champ de saisie.
- Cela permet un meilleur contrôle, une meilleure organisation et facilite la réutilisation des fonctions de gestion des événements.
- Il est courant d'avoir plusieurs gestionnaires d'événements dans un composant, chacun répondant à une action spécifique de l'utilisateur.
- Les fonctions de gestion des événements peuvent être nommées de manière descriptive pour faciliter la compréhension du code.

Gestion des événements

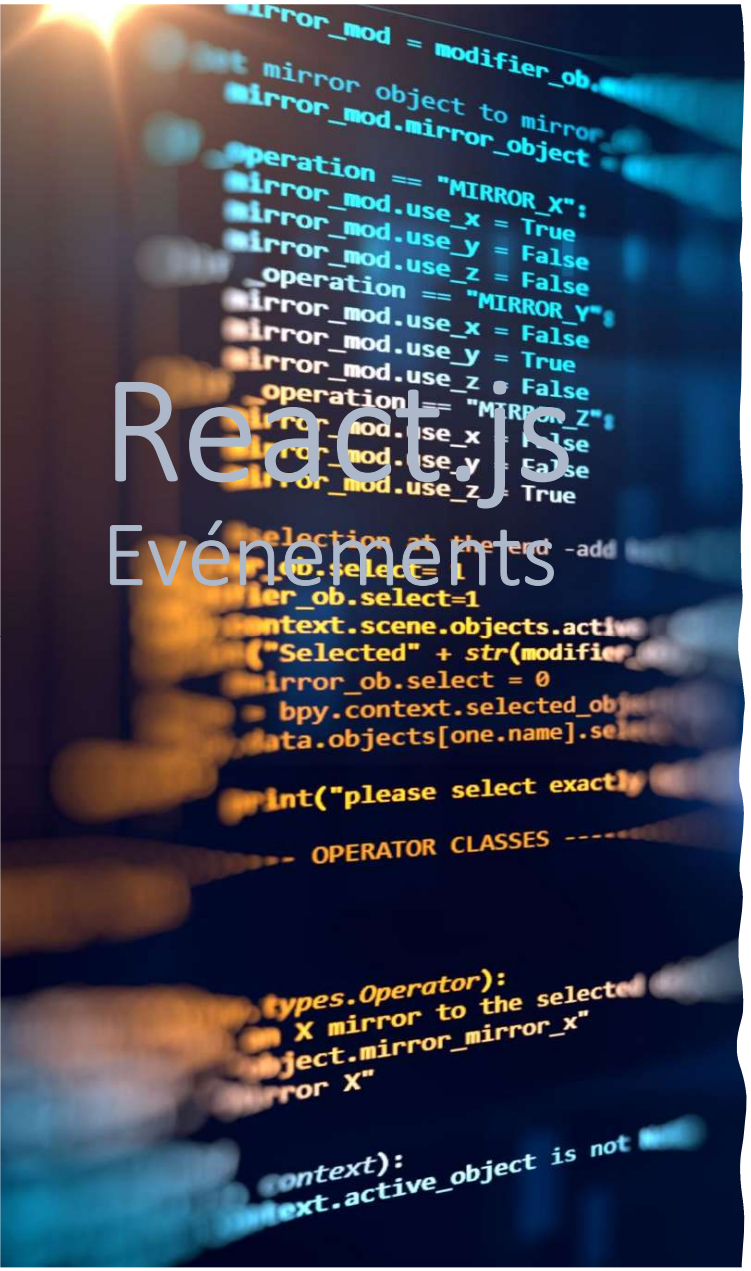
- Gestion des événements utilisateur dans les composants React.

- Exemple d'un composant avec un gestionnaire d'événements.

```
import React, { useState } from "react";

function Button() {
  const [count, setCount] = useState(0);

  const handleClick = () => {
    setCount(count + 1);
  };

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={handleClick}>Click me</button>
    </div>
  );
}
```

Gestion des événements

- Les événements dans React sont des objets JavaScript qui représentent les interactions utilisateur, tels que les clics de souris, les pressions de touches, etc.
- Les événements sont associés à des gestionnaires d'événements, qui sont des fonctions qui s'exécutent lorsque l'événement se produit.
- Dans l'exemple précédent, on a utilisé le gestionnaire d'événements **onClick** pour détecter le clic sur le bouton.



Gestion des événements

- Lorsqu'un événement se produit, React appelle le gestionnaire d'événements correspondant et exécute la fonction associée.
- Dans cet exemple, lorsque le bouton est cliqué, la fonction **handleClick** est exécutée.
- La fonction **handleClick** utilise le hook **useState** pour mettre à jour l'état local count en incrémentant sa valeur.
- Après la mise à jour de l'état, React re-renderise le composant et met à jour l'interface utilisateur pour refléter la nouvelle valeur de **count**.
- Dans l'exemple, la valeur de **count** est affichée dans l'élément <p>.
- La gestion des événements dans React suit une convention nommée "camelCase", où les noms des événements sont en minuscules et les lettres initiales des mots suivants sont en majuscules.
- Les événements couramment utilisés incluent **onClick**, **onChange**, **onSubmit**, etc.

Liste des événements couramment utilisés dans React :

Clipboard Events :

- **onCopy** : Se déclenche lors de la copie d'un élément.
- **onCut** : Se déclenche lors de la coupe d'un élément.
- **onPaste** : Se déclenche lors du collage d'un élément.

Keyboard Events :

- **onKeyDown** : Se déclenche lorsqu'une touche du clavier est enfoncée.
- **onKeyUp** : Se déclenche lorsque la touche du clavier est relâchée.

Focus Events :

- **onFocus** : Se déclenche lorsque l'élément reçoit le focus.
- **onBlur** : Se déclenche lorsque l'élément perd le focus.

Form Events :

- **onChange** : Se déclenche lorsque la valeur d'un élément de formulaire change.
- **onInput** : Se déclenche lorsque la valeur d'un élément de formulaire est modifiée.
- **onSubmit** : Se déclenche lors de la soumission d'un formulaire.

Animation Events :

- **onAnimationStart** : Se déclenche lorsque l'animation commence.
- **onAnimationEnd** : Se déclenche lorsque l'animation se termine.
- **onAnimationIteration** : Se déclenche lorsqu'une itération d'animation est terminée.

Mouse Events :

- **onClick** : Se déclenche lors d'un clic sur un élément.
- **onDoubleClick** : Se déclenche lors d'un double-clic sur un élément.
- **onMouseDown** : Se déclenche lorsque le bouton de la souris est enfoncé sur un élément.
- **onMouseUp** : Se déclenche lorsque le bouton de la souris est relâché sur un élément.
- **onMouseEnter** : Se déclenche lorsque le curseur de la souris entre dans la zone d'un élément.
- **onMouseLeave** : Se déclenche lorsque le curseur de la souris quitte la zone d'un élément.
- **onMouseMove** : Se déclenche lorsque le curseur de la souris se déplace sur un élément.
- **onMouseOver** : Se déclenche lorsque le curseur de la souris survole un élément.
- **onMouseOut** : Se déclenche lorsque le curseur de la souris quitte un élément.

Touch Events :

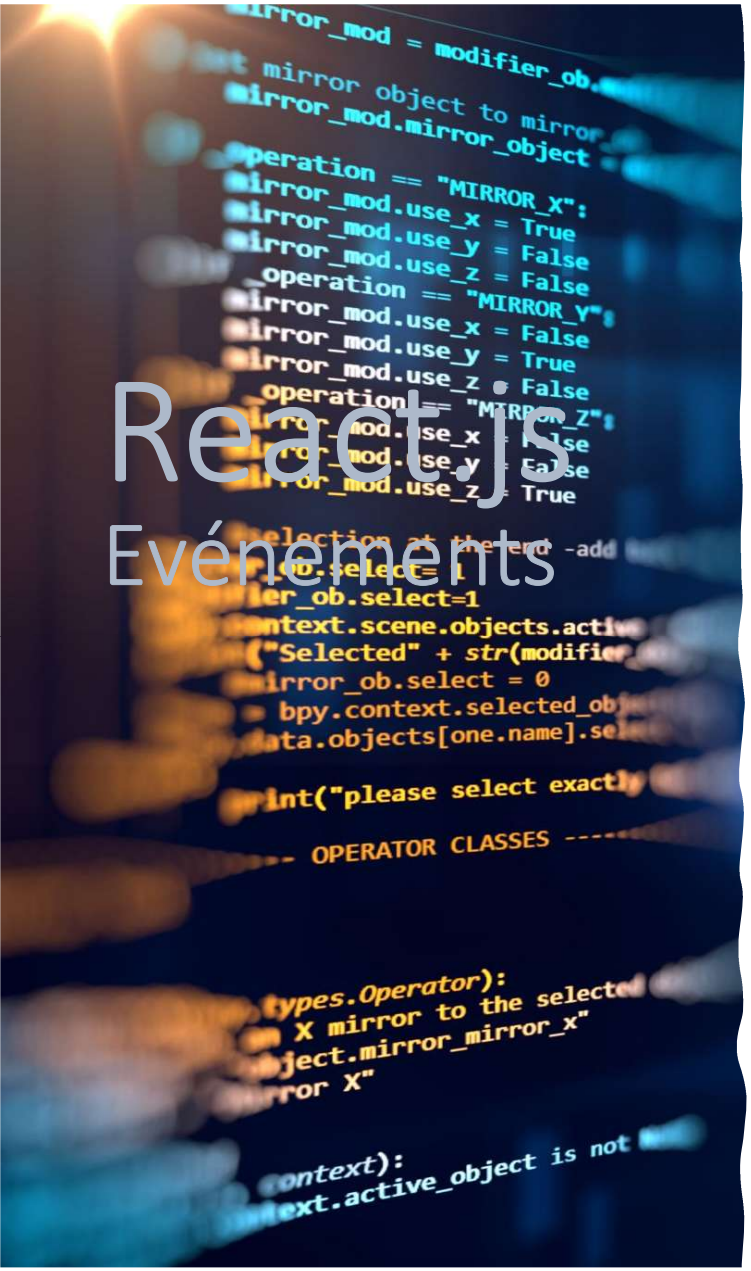
- **onTouchStart** : Se déclenche lorsqu'un écran tactile est touché.
- **onTouchMove** : Se déclenche lorsqu'un écran tactile est déplacé.
- **onTouchEnd** : Se déclenche lorsqu'un écran tactile est relâché.

Media Events :

- **onPlay** : Se déclenche lorsque la lecture d'un média commence.
- **onPause** : Se déclenche lorsque la lecture d'un média est en pause.
- **onEnded** : Se déclenche lorsque la lecture d'un média est terminée.

Transition Events :

- **onTransitionEnd** : Se déclenche lorsque la transition CSS est terminée.



Gestion des événements

Récapitulatif :

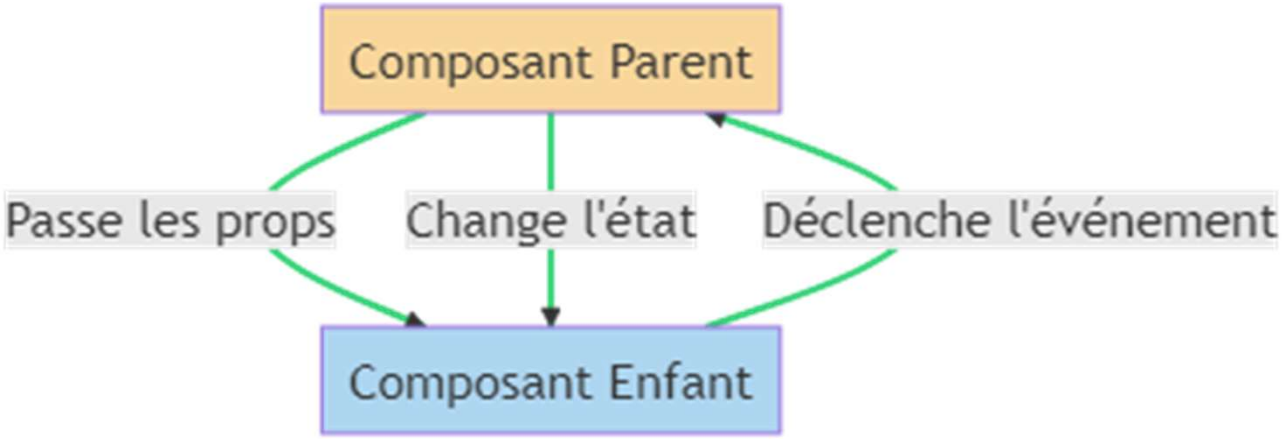
- Les événements dans React sont gérés à l'aide de gestionnaires d'événements, qui sont des fonctions associées à des événements spécifiques.
- Lorsqu'un événement se produit, React appelle le gestionnaire d'événements et exécute la fonction associée.
- Les gestionnaires d'événements peuvent utiliser le hook **useState** pour mettre à jour l'état local.
- Les mises à jour de l'état déclenchent un re-rendering du composant et mettent à jour l'interface utilisateur.

React.js

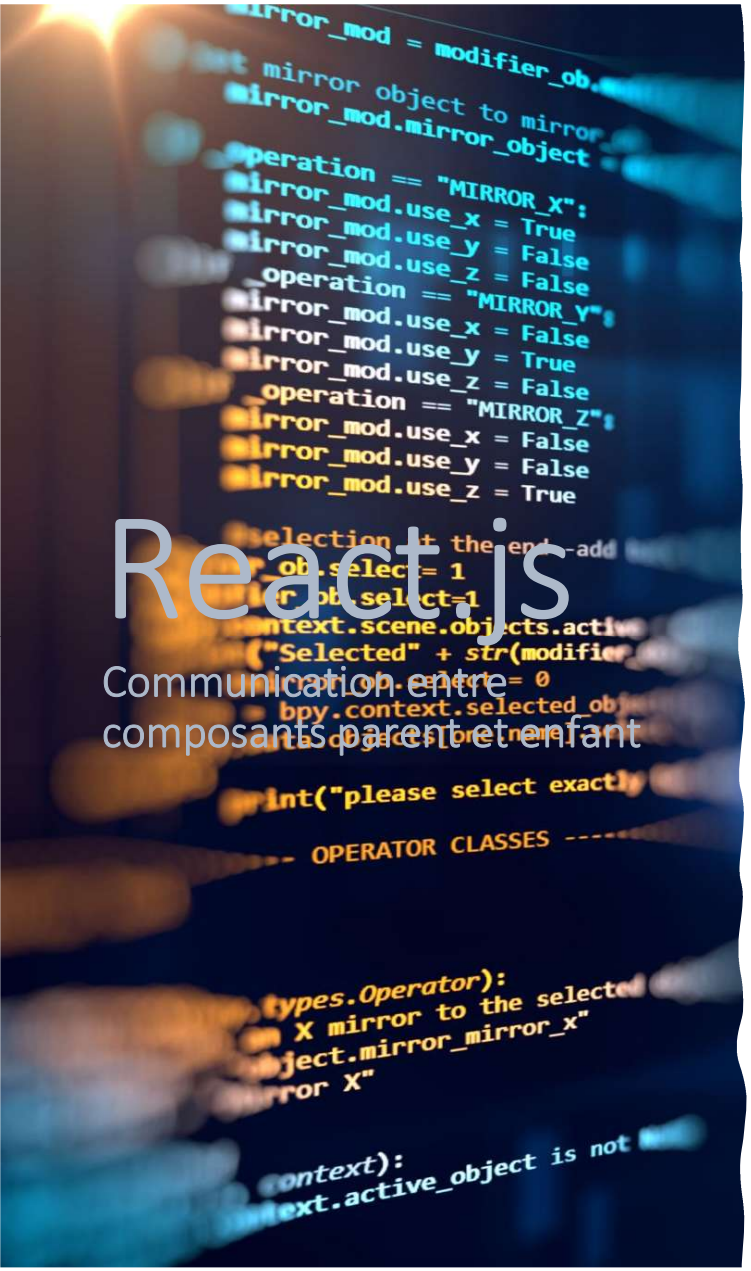
Communication entre composants parent et enfant

Communication entre composants parent et enfant

Voici un diagramme illustrant la communication entre un composant parent et un composant enfant dans React :



Dans React, les composants parent et enfant communiquent principalement par le biais de "props" et de "callbacks". Le parent passe les données à l'enfant via les "props", et l'enfant informe le parent des événements via les "callbacks".



Passage de props du parent à l'enfant

Les "**props**" (short for properties) sont un moyen pour un composant parent de passer des données à un composant enfant. Lorsque l'état du composant parent change, les props passées à l'enfant sont mises à jour, ce qui provoque le rendu de l'enfant avec les nouvelles props.

```
function ComposantParent() {
    const [nom, setNom] = useState('John');
    return <ComposantEnfant nom={nom} />;
  }

  function ComposantEnfant(props) {
    return <h1>Bonjour, {props.nom}!</h1>;
  }
```

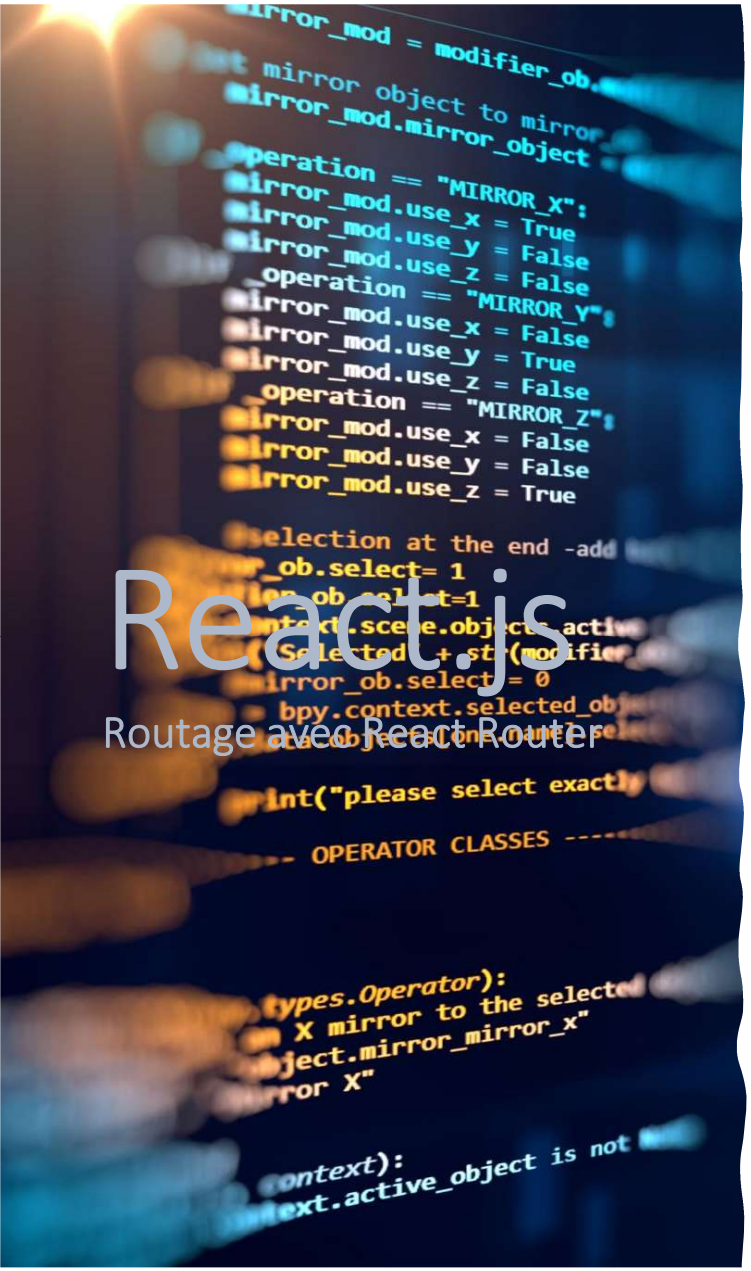
Communication de l'enfant au parent

Un composant enfant peut informer un composant parent d'un événement ou d'un changement en utilisant une fonction de rappel. Le parent passe une fonction au composant enfant via les props, et l'enfant appelle cette fonction lorsque nécessaire.

```
function ComposantParent() {
    const handleNameChange = (newName) => {
      console.log(`Le nouveau nom est ${newName}`);
    };

    return <ComposantEnfant onNameChange={handleNameChange} />;
  }

  function ComposantEnfant(props) {
    return <button onClick={() => props.onNameChange('Jane')}>Changer le nom</button>;
  }
```

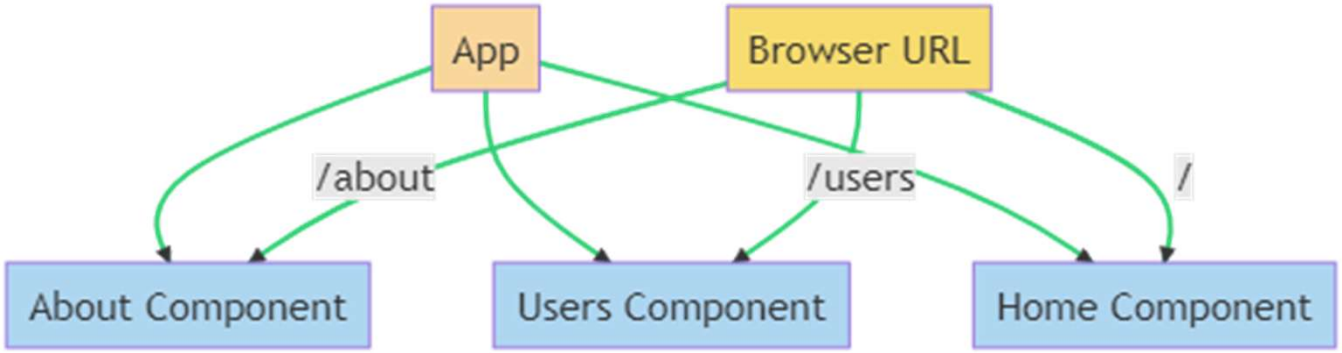
React.js

Routage avec React Router



Routage avec React Router

React Router est une bibliothèque qui permet d'ajouter du routage à votre application React. Il vous permet de définir plusieurs "routes" qui ont chacune différents composants qui sont rendus en fonction de l'URL du navigateur.



Utilisation de BrowserRouter

Pour commencer à utiliser React Router, on doit envelopper l'application dans un composant **BrowserRouter**. Ce composant est généralement placé à la racine de l'application et fournit le contexte de routage pour tous les autres composants de routage.

```
import { BrowserRouter as Router } from 'react-router-dom';

function App() {
  return (
    <Router>
      {/* Les autres composants vont ici */}
    </Router>
  );
}
```

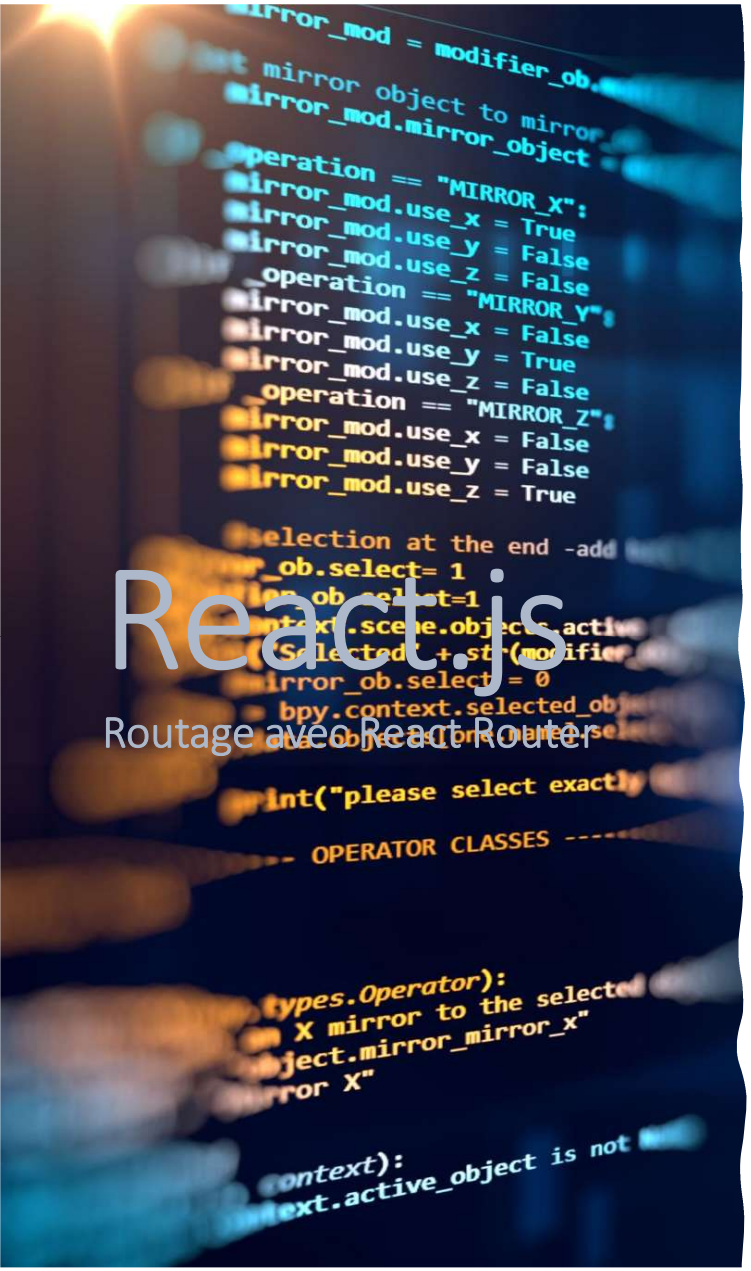
Définition des routes

Une fois que l'application est enveloppée dans un **BrowserRouter**, on peut commencer à définir des routes à l'aide du composant **Route**. Chaque **Route** a une prop **path** qui détermine quand la route doit être rendue. Si l'URL actuelle correspond au **path** de la **Route**, alors les enfants de la **Route** seront rendus.

```
import { BrowserRouter as Router, Route } from 'react-router-dom';

function App() {
  return (
    <Router>
      <Route path="/a-propos">
        <APropos />
      </Route>
      <Route path="/utilisateurs">
        <Utilisateurs />
      </Route>
      <Route path="/">
        <Accueil />
      </Route>
    </Router>
  );
}
```

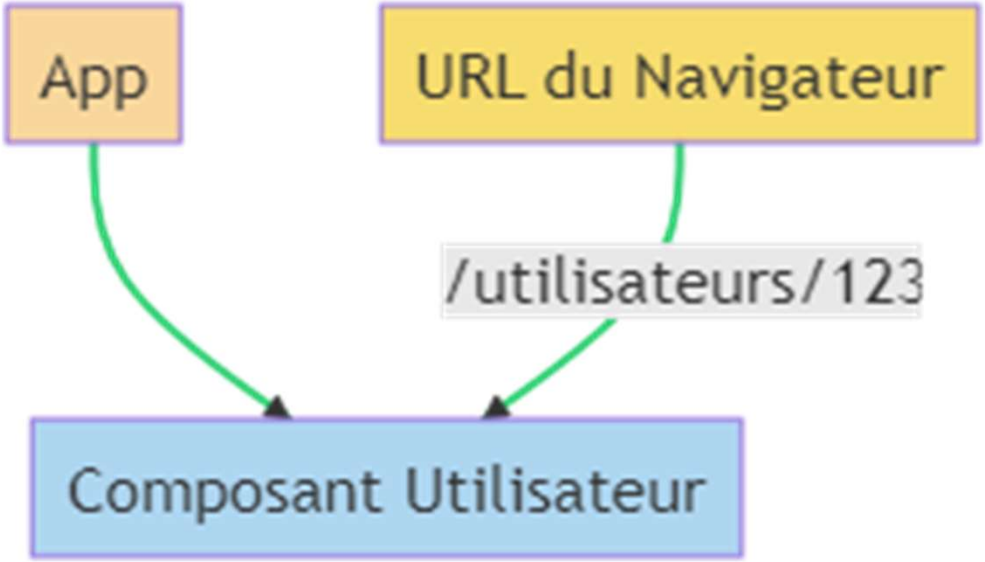
Comportement des routes

Il est important de noter que **Route** rend ses enfants lorsque l'URL correspond à son **path**, mais ne stoppe pas le rendu des autres composants. Cela signifie que si on a plusieurs **Route** avec des **path** qui correspondent à l'URL actuelle, tous ces composants seront rendus. Pour éviter cela, on peut utiliser le composant **Switch** qui rend uniquement la première **Route** ou **Redirect** qui correspond à l'emplacement actuel.



Gestion des routes et des paramètres d'URL

Les paramètres d'URL sont des valeurs dynamiques qui peuvent être incluses dans les routes. Ils sont précédés d'un deux-points (:) dans la prop **path** de la **Route**. Lorsque l'URL correspond à la route, la valeur du paramètre d'URL est extraite et peut être utilisée dans le composant.



Utilisation des paramètres d'URL

Pour utiliser un paramètre d'URL dans un composant, il est possible d'utiliser le hook **useParams** de React Router. **useParams** retourne un objet contenant les paramètres d'URL actuels.

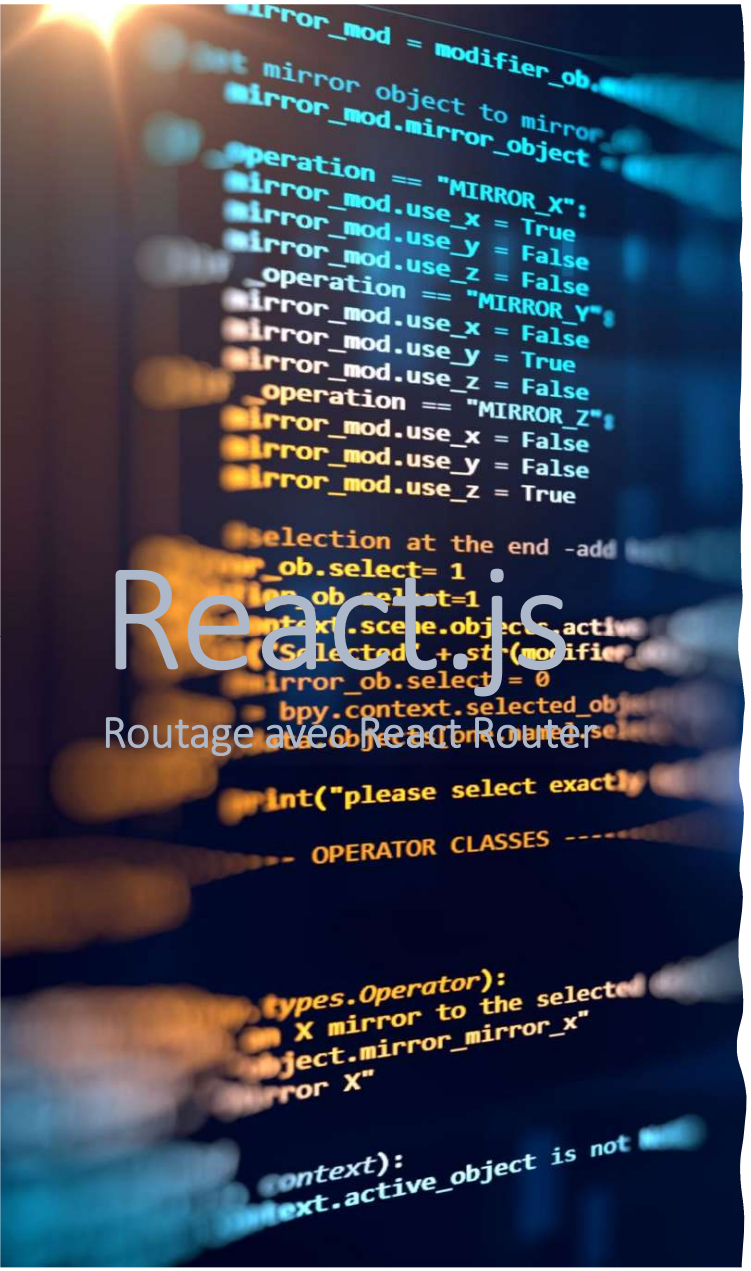
```
import { useParams } from 'react-router-dom';

function Utilisateur() {
  let { id } = useParams();
  return <h2>ID de l'utilisateur : {id}</h2>;
}
```

Exemple de route avec paramètre d'URL

Voici un exemple de route qui utilise un paramètre d'URL. Dans cet exemple, la route **/utilisateurs/:id** correspond à n'importe quelle URL qui commence par /utilisateurs/ et a une autre partie après. Cette autre partie est capturée comme le paramètre d'URL id.

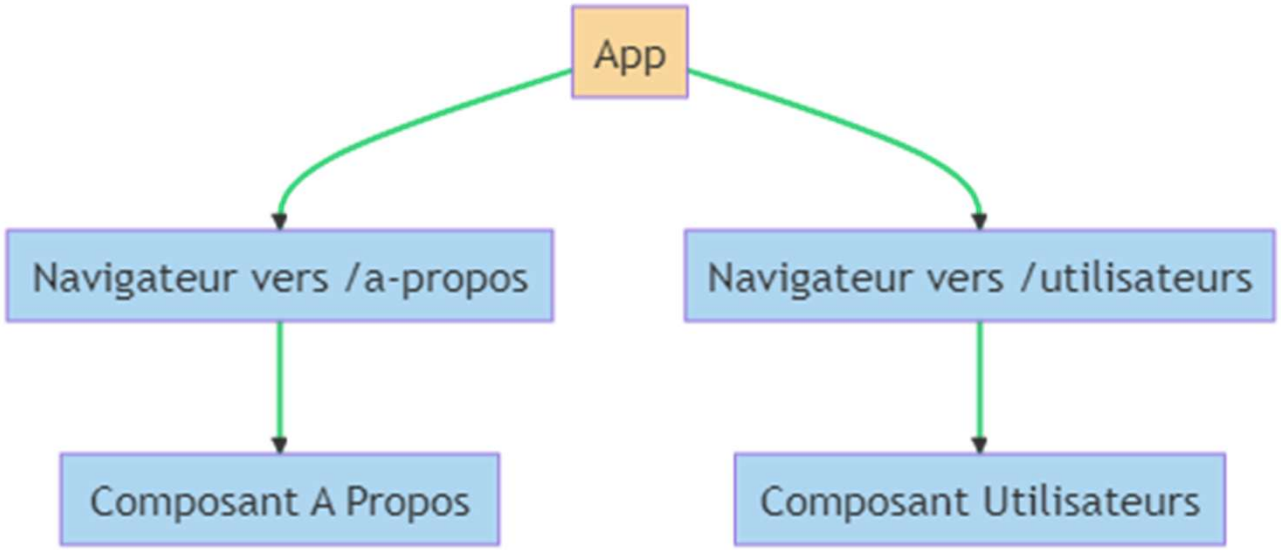
```
import { BrowserRouter as Router, Route } from 'react-router-dom';

function App() {
  return (
    <Router>
      <Route path="/utilisateurs/:id">
        <Utilisateur />
      </Route>
    </Router>
  );
}
```

Les paramètres d'URL offrent un moyen puissant de créer des routes dynamiques dans une application React. Ils permettent de capturer des valeurs de l'URL et de les utiliser dans les composants.



Introduction aux liens et à la navigation

Pour créer des liens entre différentes pages de l'application, on utilise le composant **Link** de React Router. Le **Link** a une prop **to** qui détermine l'URL vers laquelle naviguer lorsque le lien est cliqué.



Création de liens avec le composant Link

On peut créer des liens en utilisant le composant **Link** de React Router. On spécifie l'URL de destination en utilisant la prop **to**.

```
import { Link } from 'react-router-dom';

function Navigation() {
  return (
    <nav>
      <ul>
        <li>
          <Link to="/a-propos">A propos</Link>
        </li>
        <li>
          <Link to="/utilisateurs">Utilisateurs</Link>
        </li>
      </ul>
    </nav>
  );
}
```

Navigation entre les pages avec React Router

Lorsqu'un utilisateur clique sur un lien créé avec le composant **Link**, le navigateur change l'URL sans rafraîchir la page. React Router écoute les changements d'URL et rend le composant correspondant à la nouvelle route.

```
import { BrowserRouter as Router, Route, Link } from 'react-router-
dom';

function App() {
  return (
    <Router>
      <Navigation />

      <Route path="/a-propos">
        <APropos />
      </Route>
      <Route path="/utilisateurs">
        <Utilisateurs />
      </Route>
    </Router>
  );
}
```

React.js

Gestion des formulaires

Introduction aux formulaires contrôlés

Un formulaire contrôlé est un formulaire dans lequel React contrôle les entrées. Cela signifie que chaque changement d'état des éléments de formulaire est géré par React via le state.

Avantages des formulaires contrôlés :

- **Précision** : L'état du composant reflète toujours la valeur actuelle des champs de formulaire, éliminant les incohérences entre l'interface utilisateur et les données stockées.
- **Flexibilité :** Les hooks comme useState permettent de manipuler facilement les valeurs des champs à tout moment, facilitant les tâches telles que la réinitialisation des formulaires ou la pré-remplissage des données.
- **Validation** : La validation des entrées est simplifiée car vous pouvez effectuer des vérifications à chaque mise à jour de l'état, offrant des retours instantanés sur les erreurs de saisie.
- **Modularité** : Les composants fonctionnels encouragent la création de petites unités de logique réutilisables, facilitant la gestion des différents champs de formulaires comme des composants isolés.

Introduction aux formulaires contrôlés

- Le nœud "Élément de formulaire" représente un élément de formulaire dans le composant React, comme un <**input**>, <**textarea**> ou <**select**>.
- Le nœud "State de React" représente l'état du composant React. Il met à jour la valeur de l'élément de formulaire en fonction des interactions de l'utilisateur.
- Le nœud "Utilisateur" représente l'utilisateur qui interagit avec le formulaire.

Lorsque l'utilisateur interagit avec l'élément de formulaire (par exemple, en tapant dans un champ de texte), la valeur de l'élément de formulaire est mise à jour dans le state de React. Ensuite, React met à jour la valeur de l'élément de formulaire dans l'interface utilisateur pour correspondre à la valeur dans le state. C'est ce qu'on appelle un "formulaire contrôlé".

Création de formulaires contrôlés

L'idée principale derrière les formulaires contrôlés est que React doit être la **"source unique de vérité"** pour les données du formulaire. Cela signifie que l'état du composant (et donc de l'application) doit toujours être synchronisé avec l'interface utilisateur du formulaire. Cela facilite la manipulation et la validation des données du formulaire.

Exemple de code pour un formulaire contrôlé

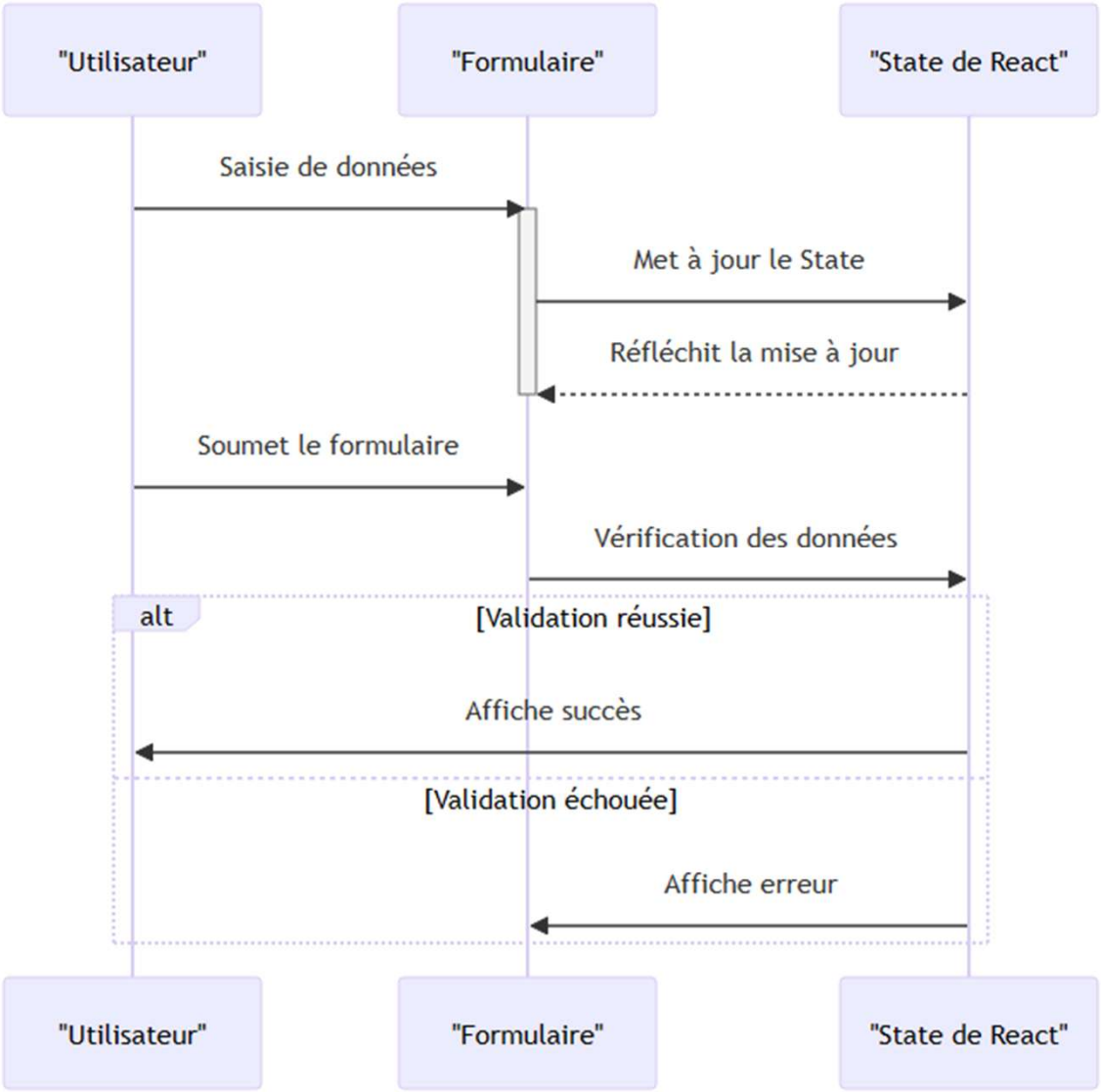
```
import React, { useState } from 'react';

function Formulaire() {
  const [nom, setNom] = useState(''); // pour créer l'état nom.

  const handleChange = (event) => {
        // met à jour l'état avec la valeur actuelle de l'input.
    setNom(event.target.value);
  };

  const handleSubmit = (event) => {
        // gère la soumission du formulaire.
    event.preventDefault();
    alert(`Le nom soumis est : ${nom}`);
  };

  return (
    <form onSubmit={handleSubmit}>
      <label>
        Nom:
        <input type="text" value={nom} onChange={handleChange} />
      </label>
      <button type="submit">Soumettre</button>
    </form>
  );
}
```

Validation des données de formulaire

Gestion de la validation des données de formulaire avec React

- React n'a pas de fonctionnalité intégrée pour la validation des formulaires, mais qu'il est possible d'implémenter facilement la validation en utilisant le state et les événements.
- Utiliser l'événement onChange pour valider les données à chaque fois que l'utilisateur modifie une valeur de formulaire.
- Stocker les erreurs de validation dans le state et les afficher à l'utilisateur.

Introduction aux formulaires contrôlés



Exemple de Validation des données de formulaire

```
function Formulaire() {
  const [value, setValue] = useState('');
  const [error, setError] = useState('');

  const handleChange = (event) => {
      const newValue = event.target.value;
      if (newValue.length < 5) {
          setError('Le texte doit avoir au moins 5 caractères');
      } else {
          setError('');
      }
      setValue(newValue);
  };

  const handleSubmit = (event) => {
      event.preventDefault();
      if (!error) {
          alert('Le texte soumis est : ' + value);
      }
  };

  return (
      <form onSubmit={handleSubmit}>
          <label>
              Nom :
              <input type="text" value={value} onChange={handleChange} />
          </label>
          {error && <p>{error}</p>}
          <input type="submit" value="Soumettre" />
      </form>
  );
}
```

Gestion de multiples champs : Utiliser un seul objet state pour gérer plusieurs champs de formulaire.

```
function Formulaire() {
  const [formData, setFormData] = useState({
    nom: '',
    age: ''
  });

  const handleChange = (event) => {
    const { name, value } = event.target;
    setFormData(prevState => ({
      ...prevState,
      [name]: value
    }));
  };

  const handleSubmit = (event) => {
    event.preventDefault();
    alert(`Nom: ${formData.nom}, Age: ${formData.age}`);
  };

  return (
    <form onSubmit={handleSubmit}>
      <label>
        Nom:
        <input type="text" name="nom" value={formData.nom} onChange={handleChange} />
      </label>
      <label>
        Age:
        <input type="number" name="age" value={formData.age} onChange={handleChange} />
      </label>
      <button type="submit">Soumettre</button>
    </form>
  );
}
```

Formulaire dynamique : Ajouter ou supprimer des champs de formulaire dynamiquement basé sur les actions de l'utilisateur.

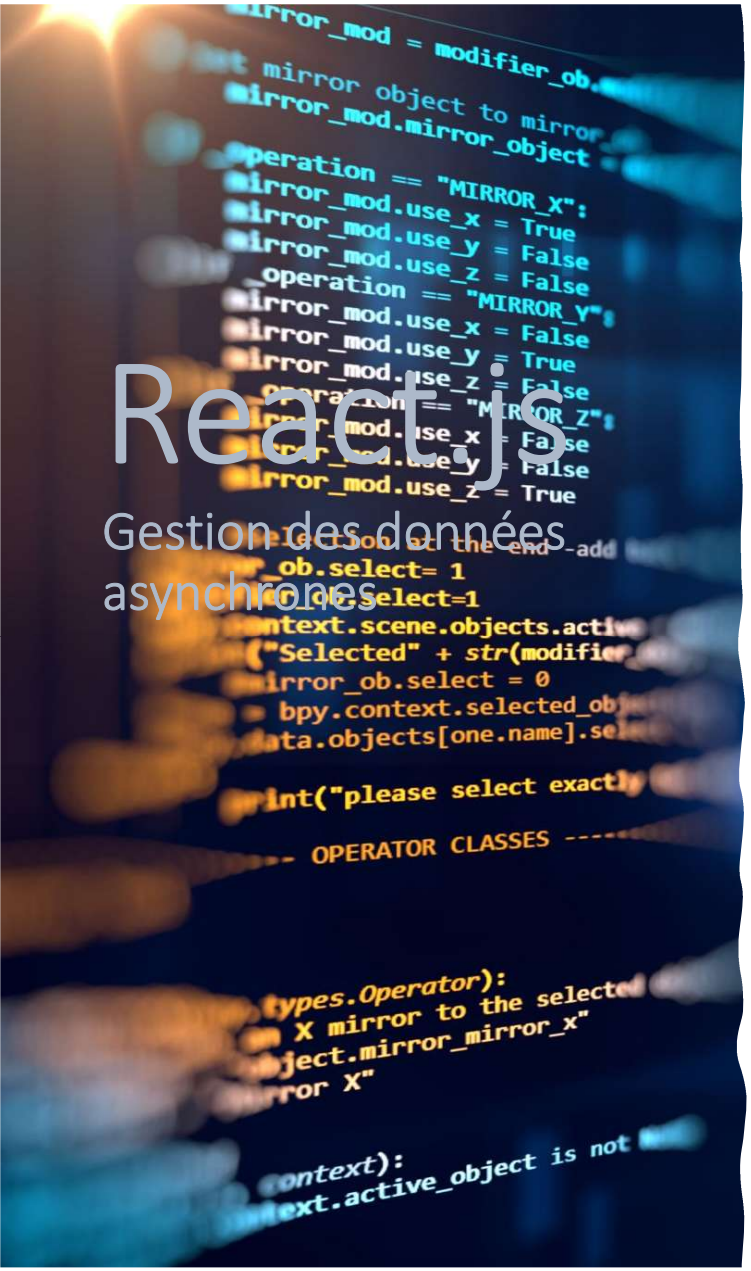
```
const [emails, setEmails] = useState(['']);

const addField = () => setEmails([...emails, '']);
const handleChange = (i, value) => {
  const newEmails = [...emails];
  newEmails[i] = value;
  setEmails(newEmails);
};
```

Intégration avec des bibliothèques tierces : Utiliser des bibliothèques comme Formik pour gérer des formulaires complexes avec moins de boilerplate.

```
import { useFormik } from 'formik';

function Formulaire() {
  const formik = useFormik({
    initialValues: { nom: '', age: '' },
    onSubmit: values => {
      alert(JSON.stringify(values, null, 2));
    }
  });

  return (
    <form onSubmit={formik.handleSubmit}>
      <label>
        Nom:
        <input type="text" {...formik.getFieldProps('nom')} />
      </label>
      <label>
        Age:
        <input type="number" {...formik.getFieldProps('age')} />
      </label>
      <button type="submit">Submit</button>
    </form>
  );
}
```

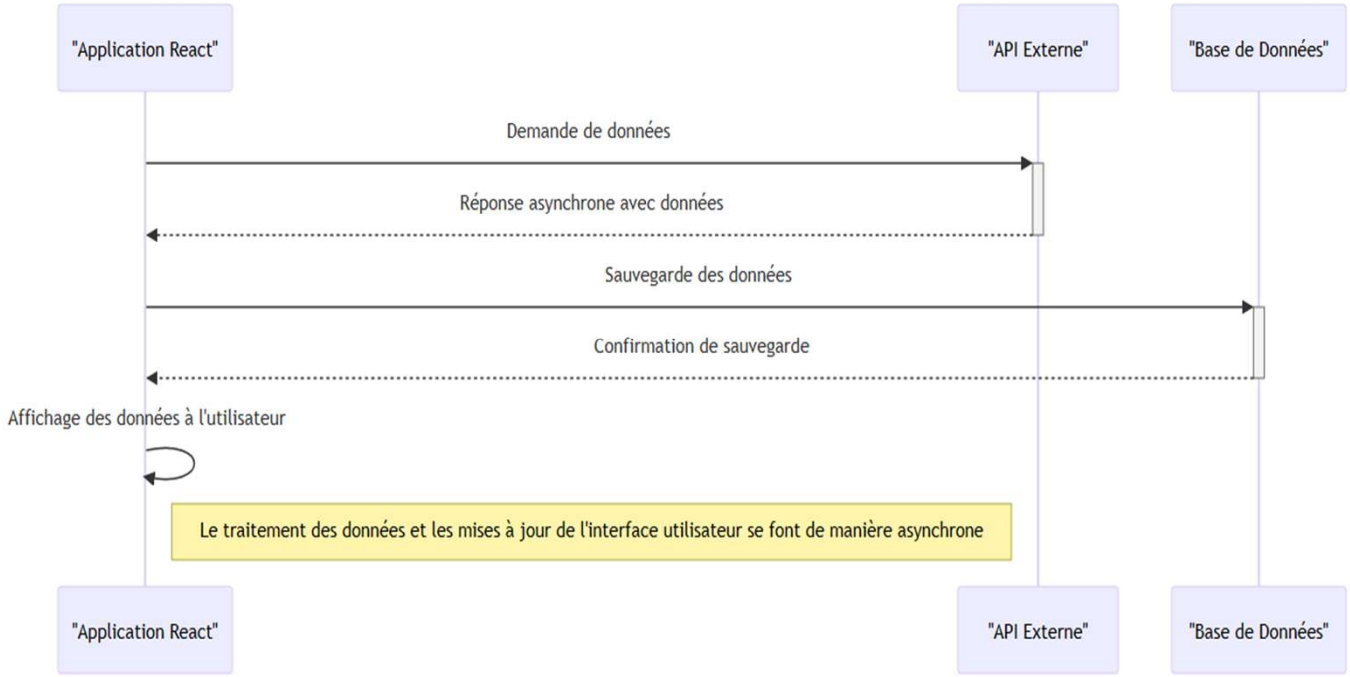
React.js

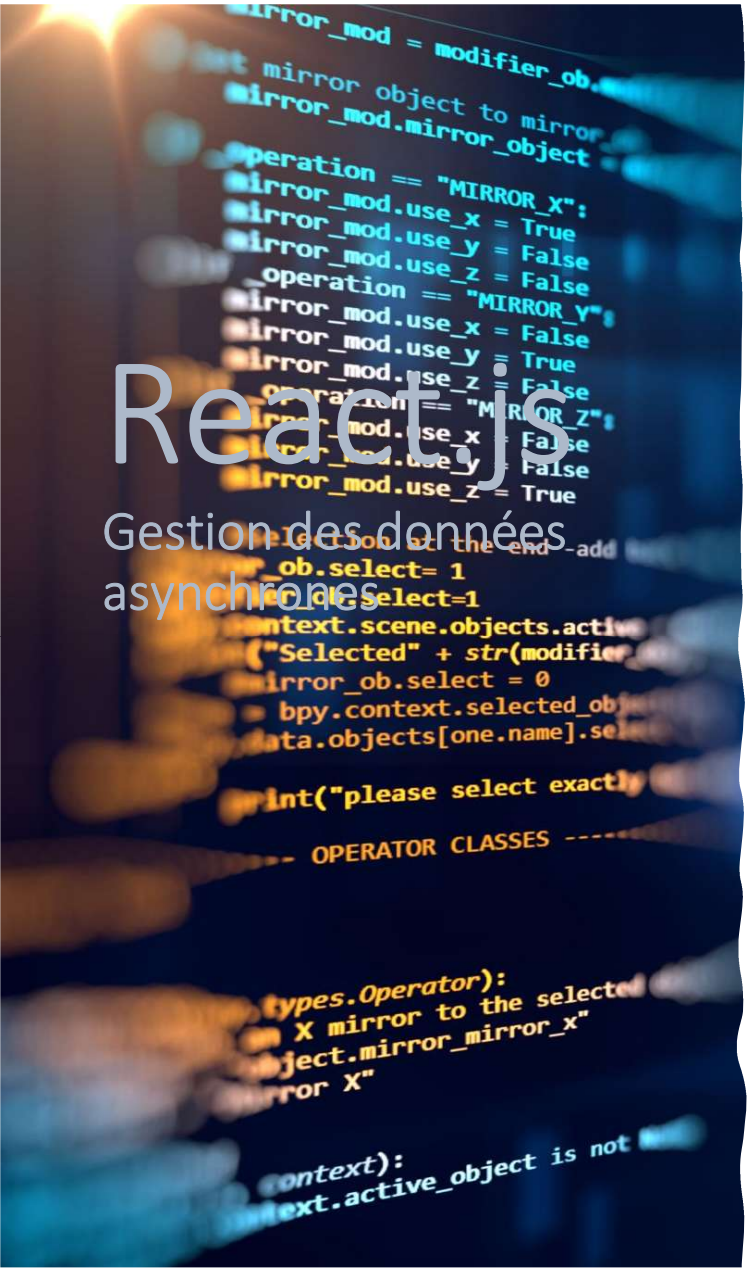
Gestion des données asynchrones



Comprendre les données asynchrones dans React

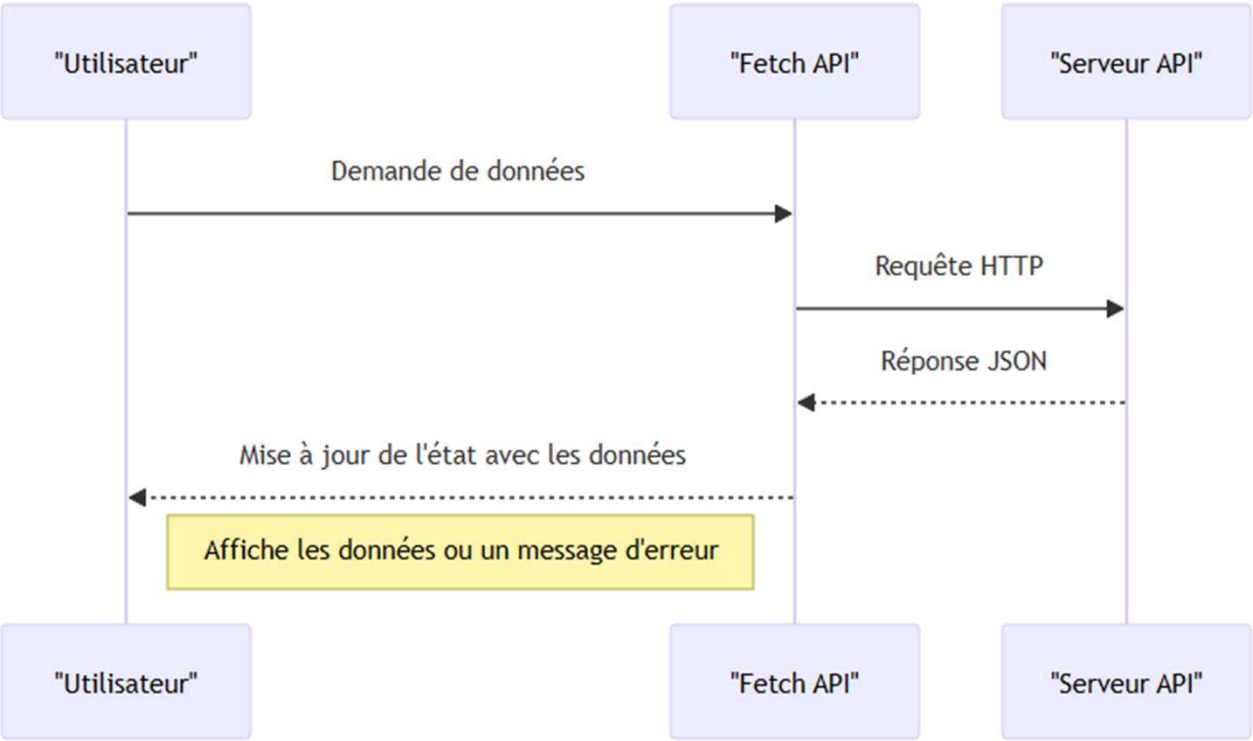
Les opérations asynchrones sont vitales dans les applications modernes car elles permettent d'améliorer l'expérience utilisateur en rendant les interactions fluides et non bloquantes. Ce type de gestion de données est essentiel pour maintenir l'efficacité de l'application tout en gérant les entrées/sorties ou les longues opérations de traitement sans interrompre l'interface utilisateur.

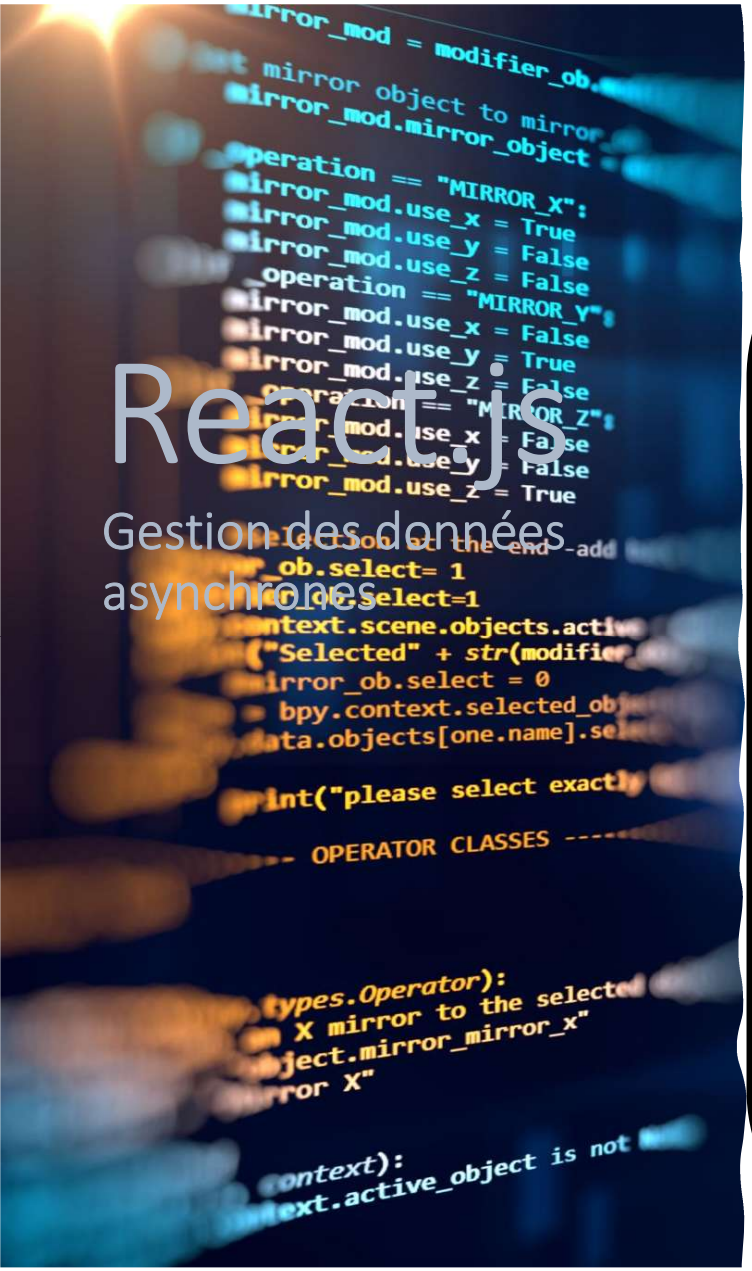




Utilisation de Fetch pour les requêtes API

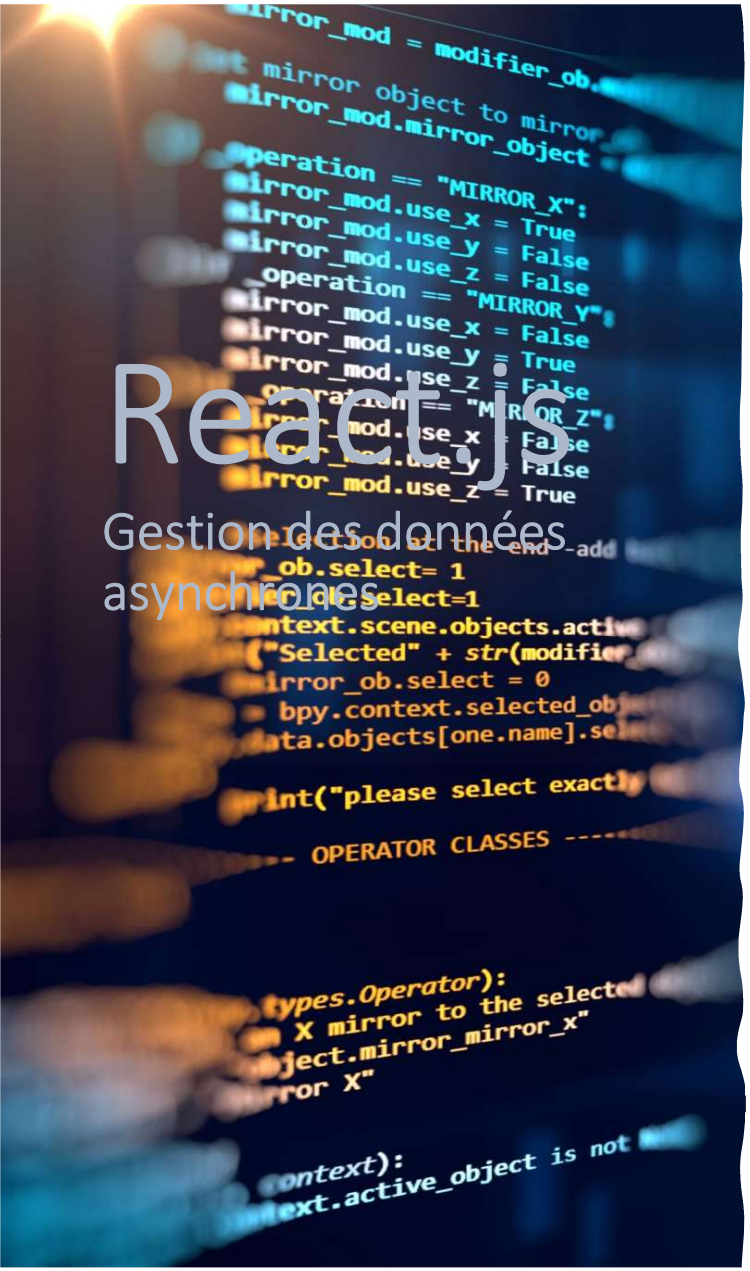
Fetch est une fonction intégrée dans la plupart des navigateurs modernes pour faire des requêtes HTTP. Il retourne une promesse qui résout en réponse à une requête, que vous pouvez ensuite convertir en JSON.





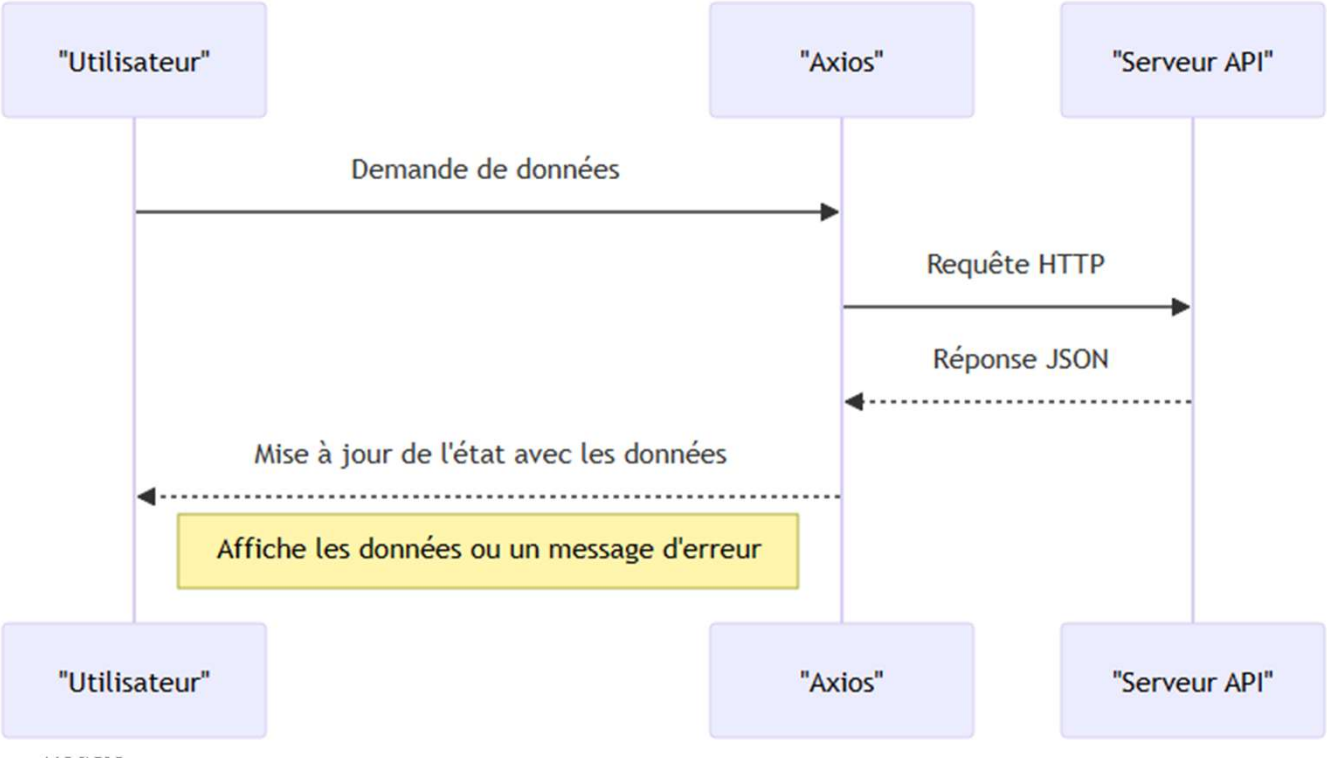
Utilisation de Fetch pour les requêtes API

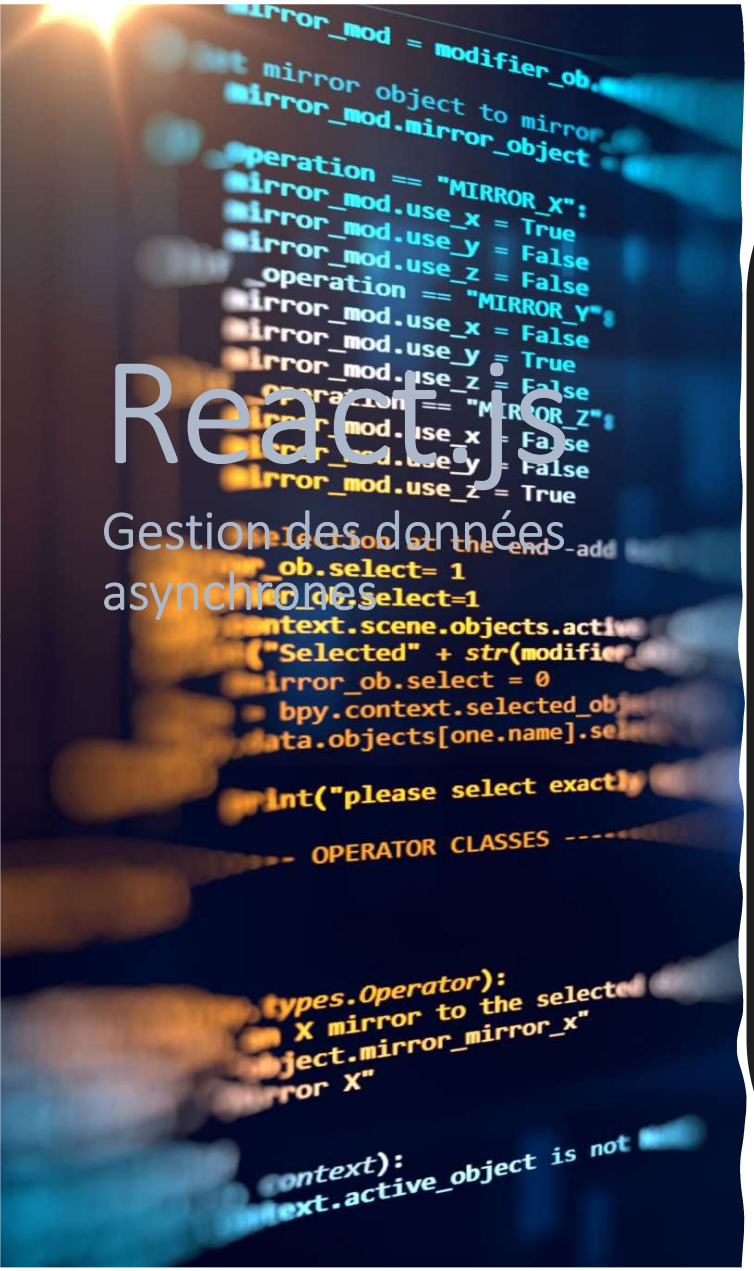
```
useEffect(() => {
  const ctrl = new AbortController();
  async function fetchData() {
    try {
      setLoading(true);
      const res = await fetch('https://api.example.com/data', { signal: ctrl.signal });
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      const json = await res.json();
      setData(json);
    } catch (err) {
      if (err.name !== 'AbortError') setError(err);
    } finally {
      setLoading(false);
    }
  } fetchData();
  return () => ctrl.abort();
}, []);
```



Utilisation d'Axios pour les requêtes API

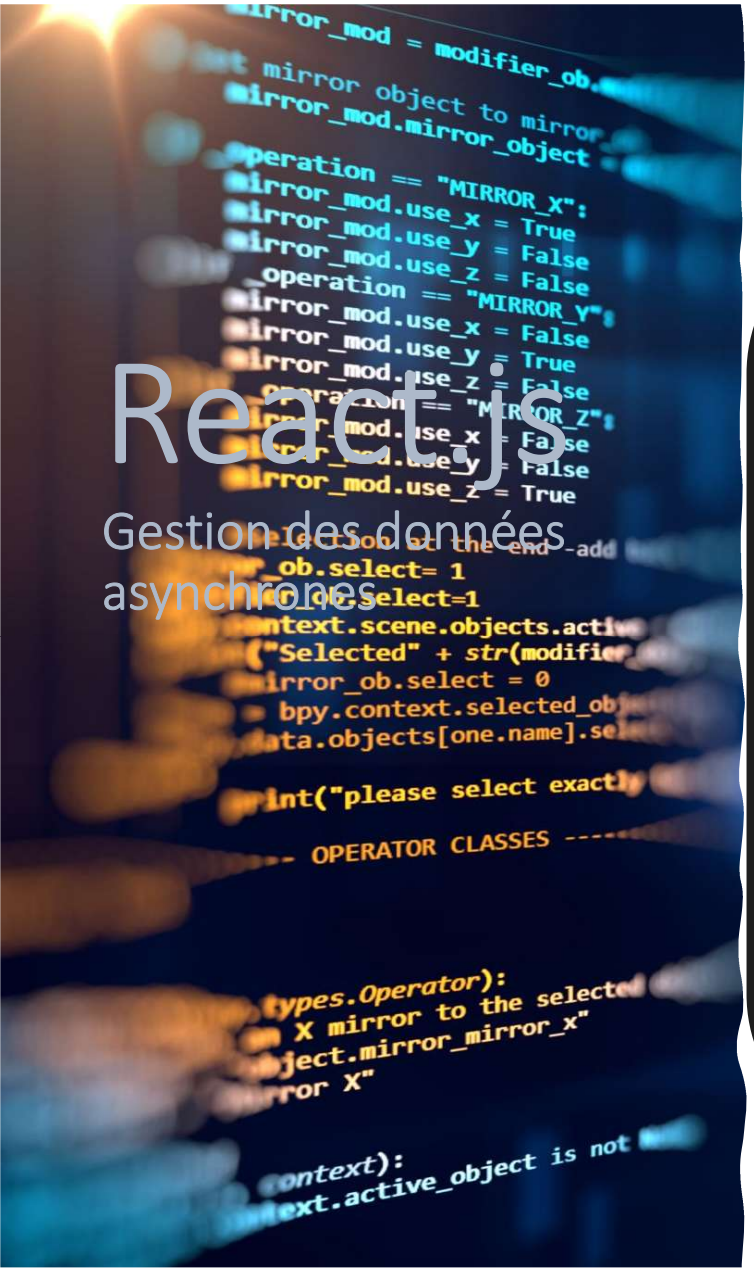
Axios est une bibliothèque tierce qui offre quelques fonctionnalités supplémentaires par rapport à **Fetch**, comme l'interception des requêtes et des réponses, et la possibilité d'annuler des requêtes.





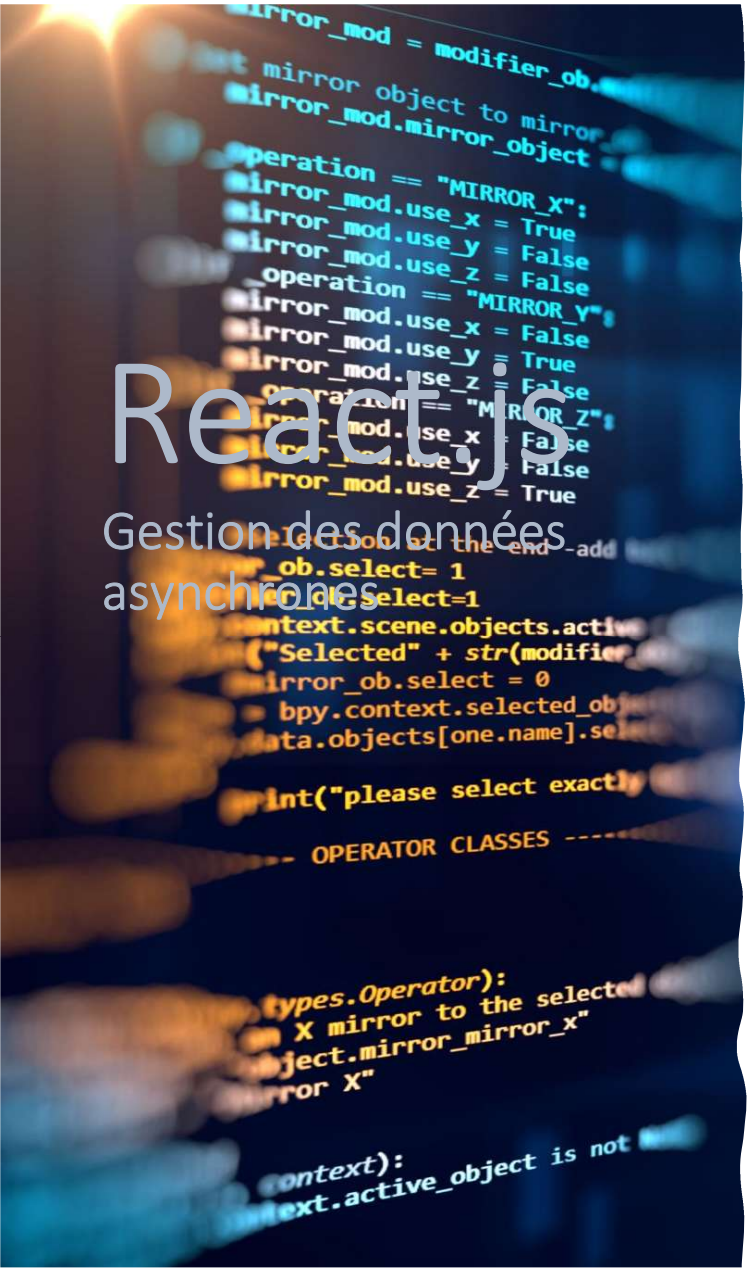
Utilisation d'Axios pour les requêtes API

```
useEffect(() => {
  const ctrl = new AbortController();
  const api = axios.create({ baseURL: 'https://api.example.com', timeout: 8000 });
  (async () => {
    try {
      setLoading(true);
      const res = await api.get('/data', { signal: ctrl.signal });
      setData(res.data);
    } catch (err) {
      if (err.name !== 'CanceledError') setError(err);
    } finally {
      setLoading(false);
    }
  })();
  return () => ctrl.abort();
}, []);
```



Interception des requêtes et des réponses

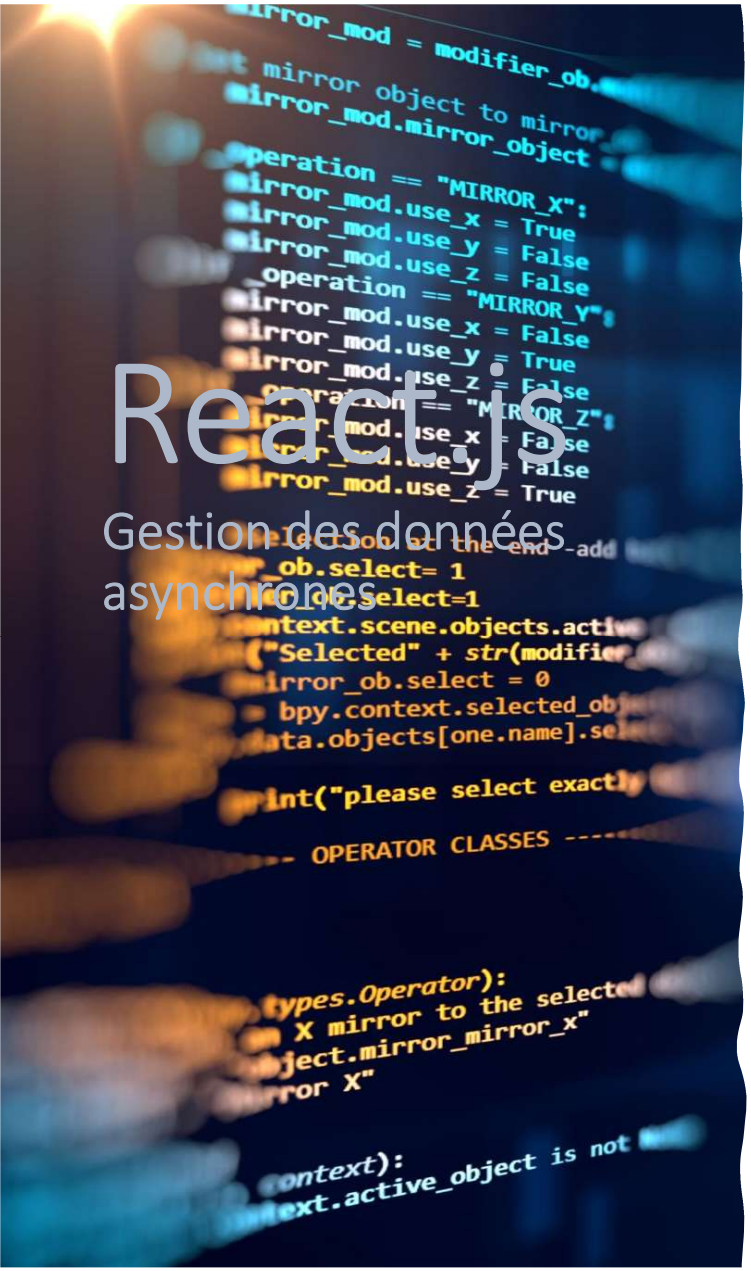
```
useEffect(() => {
  const reqId = api.interceptors.request.use((config) => {
    console.log('➡️ Request', config.url);
    return config;
  });
  const resId = api.interceptors.response.use(
    (response) => response,
    (error) => Promise.reject(error)
  );
  return () => {
    api.interceptors.request.eject(reqId);
    api.interceptors.response.eject(resId);
  };
}, []);
```



Transformation des données (POST)

```
axios.post('/login', {
  username: 'John Doe',
  password: 's3cr3t'
}, {
  transformRequest: [(data, headers) => {
    // transforme les données JSON en une chaîne de requête
    // URL-encodée avant l'envoi.
    const transformedData = Object.entries(data).map(([key, value]) =>
`${encodeURIComponent(key)}=${encodeURIComponent(value)}`).join('&');
    headers['Content-Type'] = 'application/x-www-form-urlencoded';
    return transformedData;
  }]
});
```

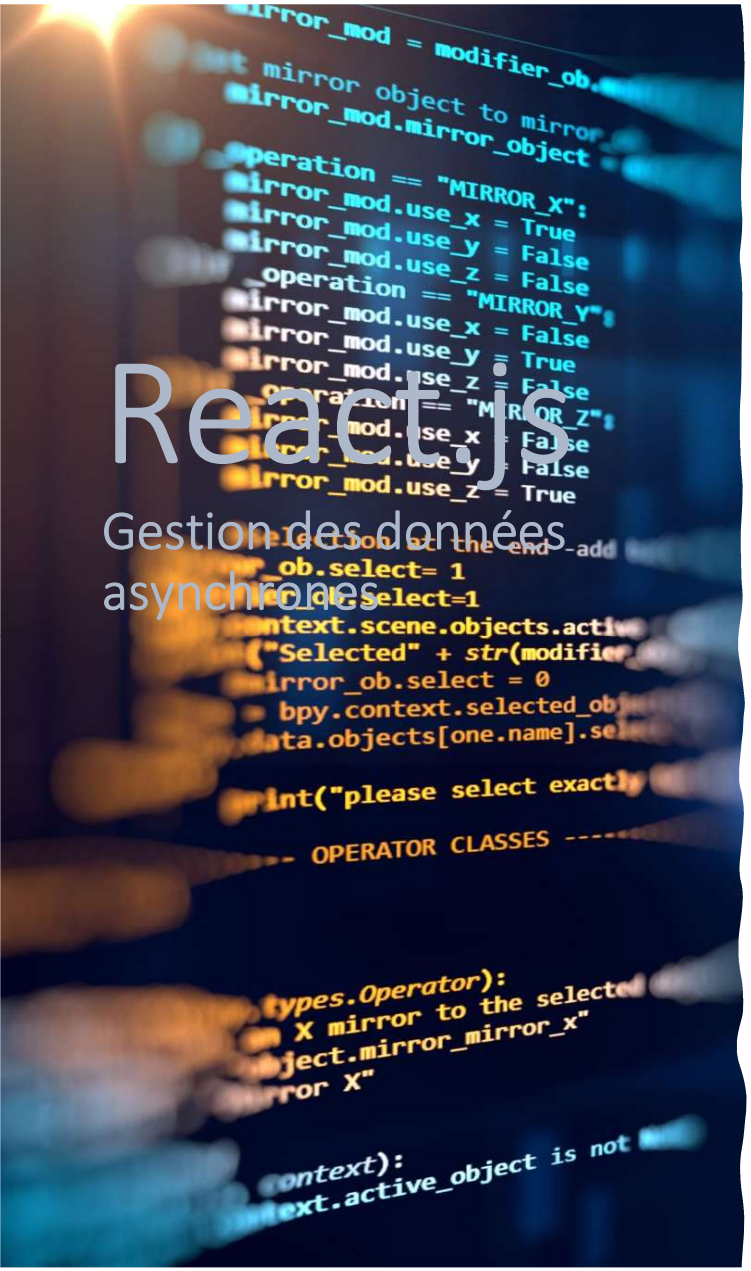
L’encodage application/x-www-form-urlencoded permet d’envoyer les données dans le même format qu’un formulaire HTML classique. Chaque paire clé=valeur est transformée en texte compatible HTTP (**username=John%20Doe&password=s3cr3t**), garantissant que les caractères spéciaux (espaces, &, =…) ne perturbent pas la requête.Ce format reste très utilisé pour les requêtes d’authentification ou les API héritées qui ne gèrent pas encore le JSON.



Gestion des erreurs (Axios)

Différencier erreurs serveur (4xx/5xx), réseau (pas de réponse), et autres (config)
Afficher un message utile à l'utilisateur + logs dev

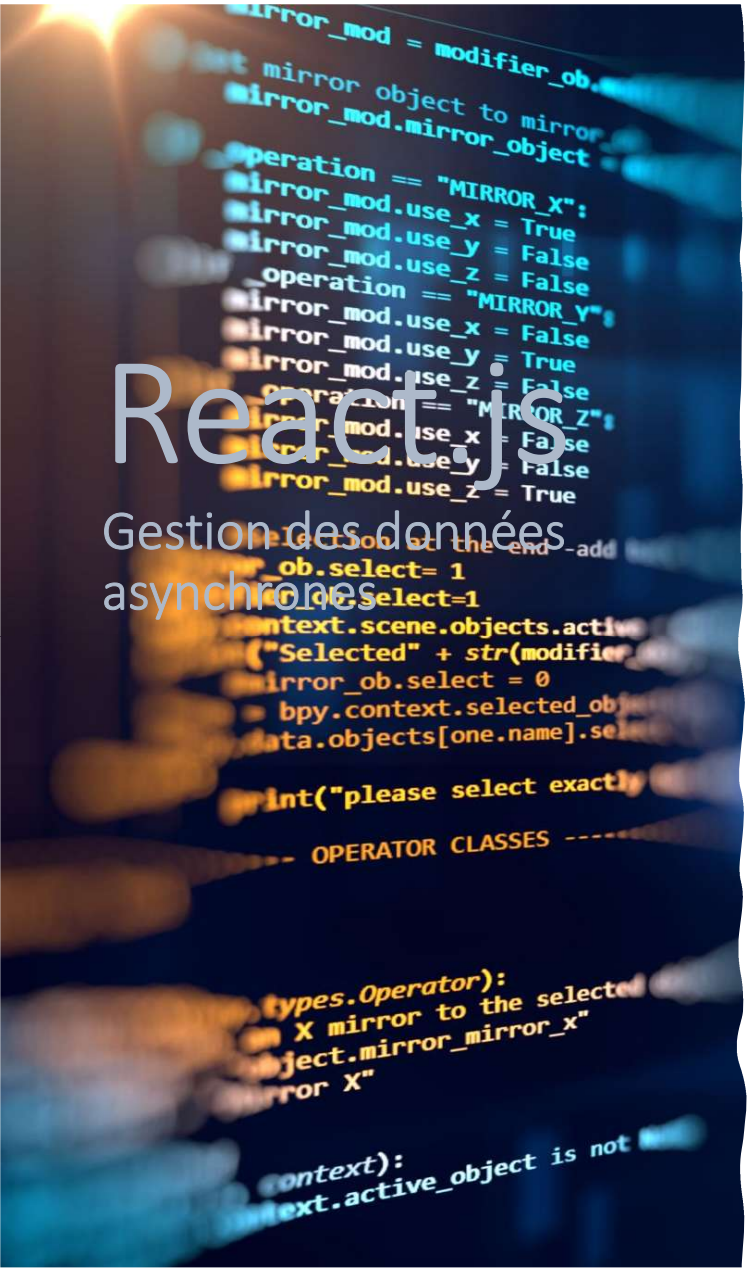
```
axios.get('/user/12345')
.then(response => console.log(response))
.catch(error => {
  if (error.response) {
    // La demande a été faite et le serveur a répondu avec un statut
d'erreur
    console.error('Error', error.response.status);
    console.error('Error data', error.response.data);
  } else if (error.request) {
    // La demande a été faite mais aucune réponse n'a été reçue
    console.error('No response received');
  } else {
    // Quelque chose s'est mal passé lors de la configuration de la
requête
    console.error('Error', error.message);
  }
});
```



Gestion des temps d'attente

Timeout côté client (Axios) ≠ timeout côté serveur
Pour Fetch, implémenter

```
axios.get('/user/12345', {
  timeout: 5000  // Temps d'attente en millisecondes
})
.then(response => console.log(response))
.catch(error => console.error('Timeout or Network Error', error));
```

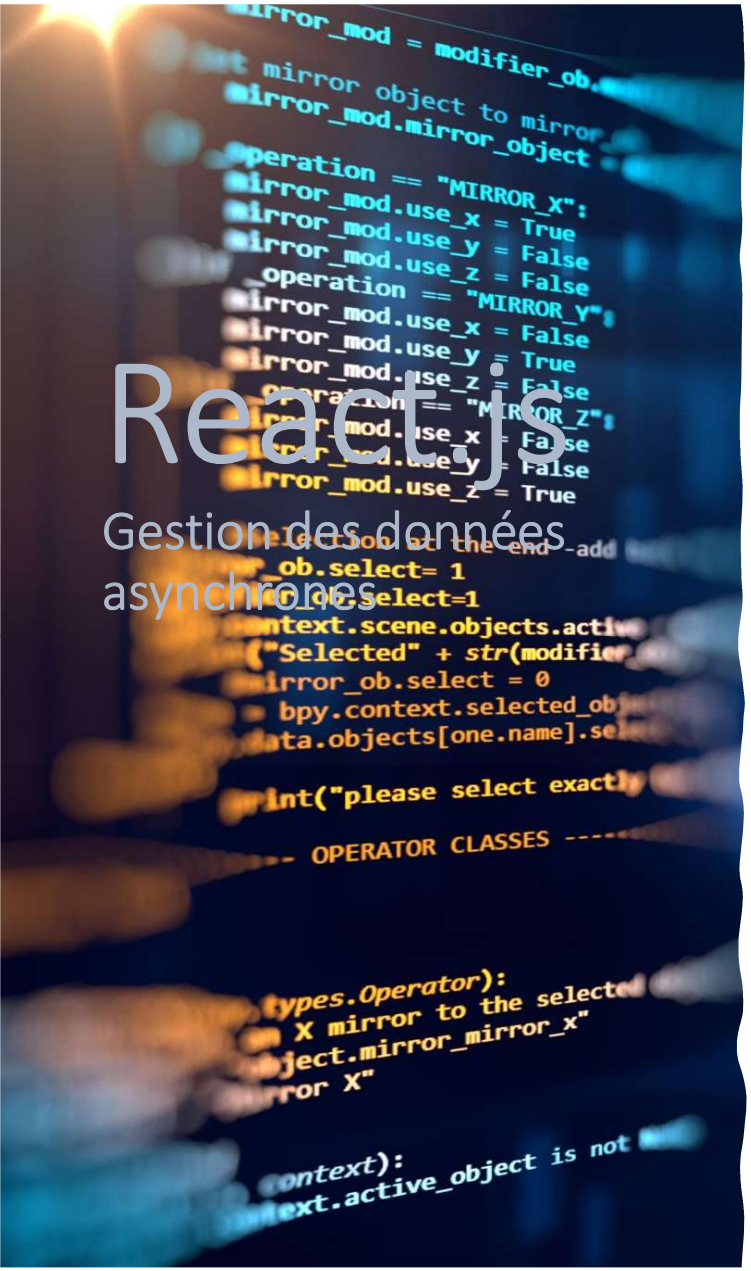


Maîtriser useEffect pour les opérations asynchrones

Le **hook useEffect** permet d'effectuer des effets secondaires dans les composants fonctionnels. Dans le contexte des opérations asynchrones, il est souvent utilisé pour exécuter des requêtes API après le rendu initial d'un composant, et pour réagir à des changements spécifiques de props ou d'état.

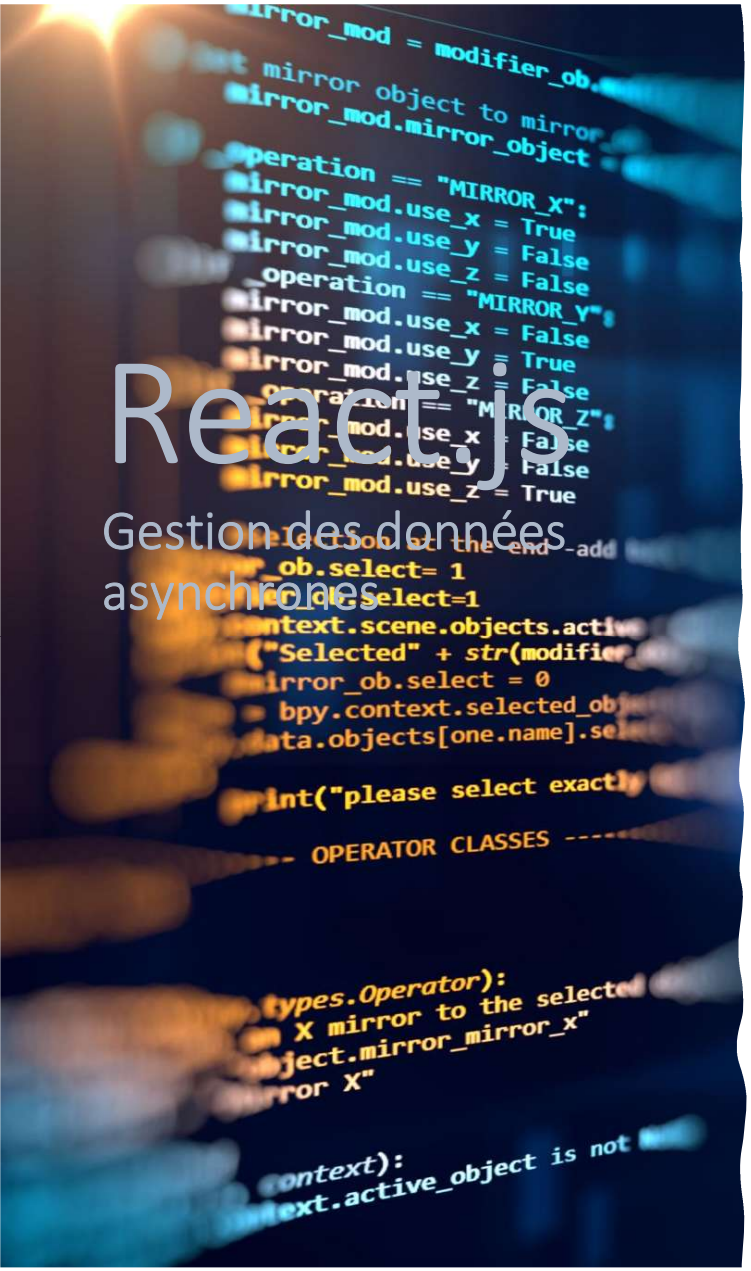
- **Dépendances de useEffect:** Permet de contrôler quand l'effet doit s'exécuter. Si le tableau de dépendances est vide ([]), l'effet s'exécute une seule fois après le premier rendu. Si le tableau contient des valeurs, l'effet s'exécute chaque fois que ces valeurs changent.

- **Nettoyage dans useEffect:** Parfois, des nettoyages sont nécessaires pour annuler des abonnements, des temporisateurs ou d'autres effets lorsque le composant est sur le point d'être démonté ou avant que l'effet ne se réexécute. On utilise une fonction de retour dans useEffect pour ce nettoyage.



Maîtriser useEffect pour les opérations asynchrones

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';
function AxiosAsyncExample() {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const fetchData = async () => {
      try {
        const response = await axios.get('https://api.exemple.fr/donnees');
        setData(response.data);
        setLoading(false);
      } catch (error) {
        console.error('Erreur:', error);
        setError(error);
        setLoading(false);
      }
    };
    fetchData();
  }, []);

  if (loading) return <p>Chargement en cours...</p>;
  if (error) return <p>Erreur: {error.message}</p>;
  return (
    <div>
      <h1>Données chargées</h1>
      {data && <pre>{JSON.stringify(data, null, 2)}</pre>}
    </div>
  );
}
```



Gérer des états complexes avec useReducer et Axios

```
import React, { useReducer, useEffect } from 'react';
import axios from 'axios';
const initialState = { loading: true, error: null, data: null };
function reducer(state, action) {
  switch (action.type) {
    case 'FETCH_SUCCESS':
      return { loading: false, error: null, data: action.payload };
    case 'FETCH_ERROR':
      return { loading: false, error: action.payload, data: null };
    default:
      return state;
  }
}

function ReducerAxiosExample() {
  const [state, dispatch] = useReducer(reducer, initialState);
  useEffect(() => {
    axios.get('https://api.exemple.fr/donnees')
      .then(response => dispatch({ type: 'FETCH_SUCCESS', payload: response.data }))
      .catch(error => dispatch({ type: 'FETCH_ERROR', payload: error }));
  }, []);

  const { loading, error, data } = state;
  if (loading) return <p>Chargement en cours...</p>;
  if (error) return <p>Erreur: {error.message}</p>;
  return (
    <div>
      <h1>Données chargées</h1>
      {data && <pre>{JSON.stringify(data, null, 2)}</pre>}
    </div>
  );
}
```